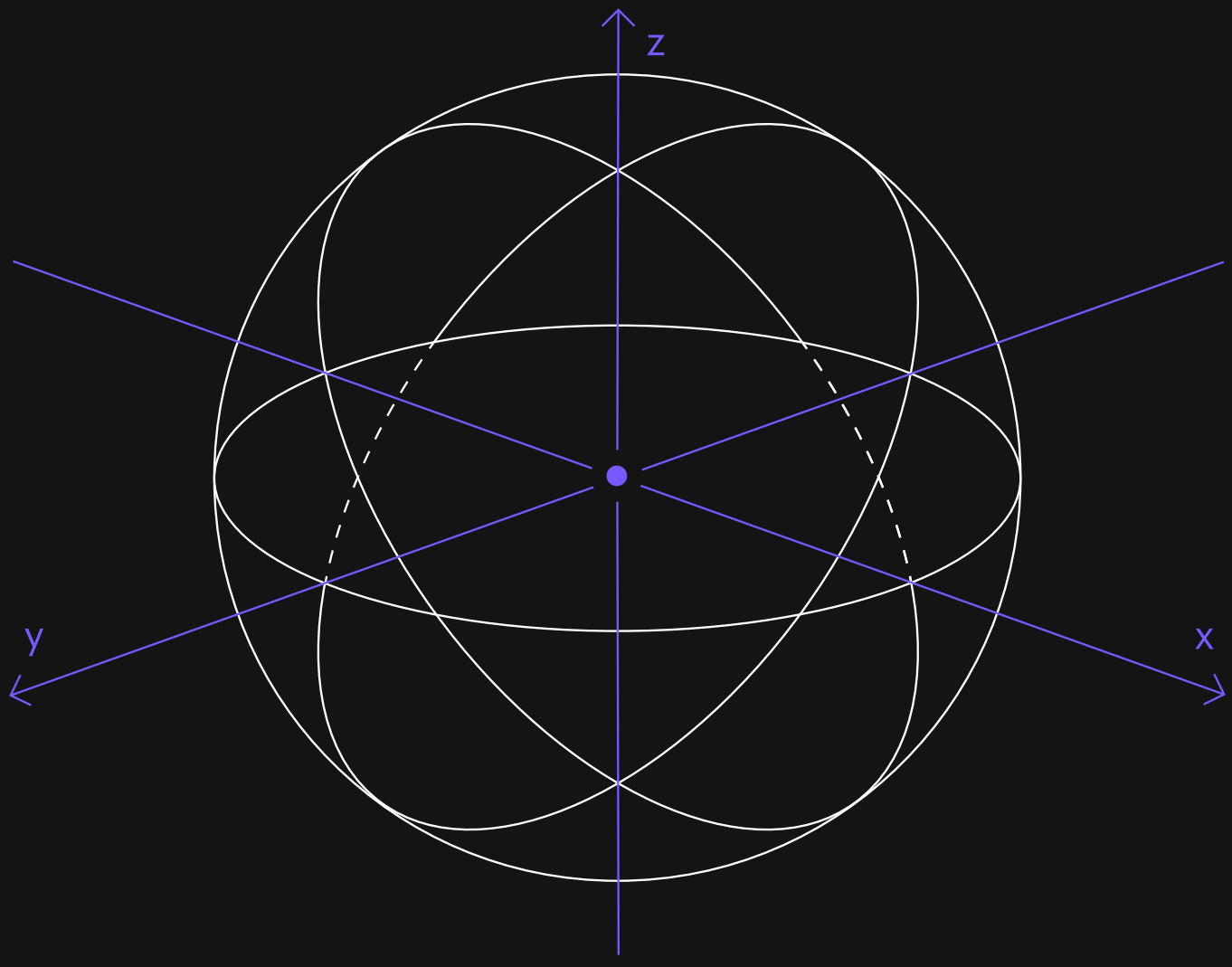
Машинное обучение

Лонгрид

Неделя 5

Логистическая регрессия, SVM



Содержание

Глава 1. Задача классификации 3
1.1. Подтипы задач классификации 4
1.2. Основные математические концепции задачи классификации 6
1.2.1. Отступ 8
1.3. Функции потерь 8
Глава 2. Логистическая регрессия 12
2.1. Функция сигмоиды 13
2.2. Почему именно сигмоида? 13
2.3. Вероятностная интерпретация логистической регрессии 14
2.4. Цель обучения: максимизация правдоподобия 14
2.5. Логарифмическая функция потерь 15
2.5.1. Связь с бинарной кросс-энтропией 16
2.6. Интерпретация предсказанных вероятностей 16
2.7. Применение логистической регрессии 18
2.8. Итоги логистической регрессии 19
Глава 3. SVM 21
3.1. Почему «векторы» 21
3.2. Почему векторы «опорные» 22
3.3. Вывод формулы отступа для SVM 23
3.4. Нормировка и опорные векторы 24
3.5. Задача оптимизации для линейно разделимого случая 26
3.6. SVM для линейно неразделимых данных 26
Глава 4. Ядерный трюк (kernel trick) 28
4.1. Исходная задача SVM 29
4.2. Переход к двойственной задаче 29
4.2.1. Предсказания 30
4.2.2. Ядерный трюк 30
4.3. Ядерные функции 31
4.3.1. Полиномиальное ядро (polynomial kernel) 31
4.3.2. Гауссово (RBF) ядро 32
Глава 5. Практическое применение и сравнение моделей 33
5.1. Применение SVM 33
5.2. Отличия SVM и логистической регрессии 33
5.3. Почему SVM не выдаёт вероятности напрямую 35
Глава 6. Калибровка вероятностей 36
6.1. Сигмоидная калибровка Платта (Platt scaling) 38
6.2. Изотоническая регрессия 39
6.3. Метрики оценки приёмов калибровки вероятностей 41
6.4. Применение калибровки 42
Если кратко 43

Логистическая регрессия, SVM

Контекст

Машинное обучение решает задачи разного типа: в одних случаях нужно предсказать число (регрессия), в других — определить, к какому классу относится объект (классификация). Логистическая регрессия и SVM — два классических подхода к задаче классификации. Они по-разному определяют границу между классами: логистическая регрессия строит границу, исходя из вероятностей принадлежности к классам, тогда как SVM ищет наиболее чёткое разделение между ними.

Применение

Логистическую регрессию применяют в задачах, где важно оценить вероятность события, например в банковских, страховых и медицинских отраслях.
SVM в индустрии применяется реже, но может быть полезен при работе с небольшими объёмами данных и хорошо разделимыми классами.

Краткий план

→ Задача классификации
→ Логистическая регрессия
→ SVM
→ Ядерный трюк (kernel trick)
→ Практическое применение и сравнение моделей
→ Калибровка вероятностей
→ Если кратко

Глава 1. Задача классификации

На прошлых занятиях мы обучали модель предсказывать числовые значения, например стоимость квартиры на основе её площади или температуру по данным датчиков. Это линейная регрессия — алгоритм, который отвечает на вопрос «Сколько?».

Но в жизни часто нужно отвечать на вопрос «Что?» или «Какой?».

- Это спам или важное письмо?
- Болеет ли пациент гриппом или обычной простудой?
- Купит клиент товар или нет?

В таких случаях модель должна не предсказать числовое значение, а выбрать категорию. Это и есть классификация.

ОПРЕДЕЛЕНИЕ

Классификация — это задача машинного обучения, в которой алгоритм учится относить объекты к одному из заранее определённых классов (категорий) на основе их признаков.

ПРИМЕР

Банк получает заявку на кредит от клиентов. У каждого клиента есть характеристики: возраст, доход, кредитная история, наличие долгов. Задача банка —

> принять решение: выдать кредит или отказать или отнести в группу «надёжный» или «рискованный». Например, можно задать следующие условия:
>
> - зарплата 60 000 ₽ + идеальная кредитная история → «одобрить»;
> - зарплата 25 000 ₽ + три просрочки → «отклонить».

1.1. ПОДТИПЫ ЗАДАЧ КЛАССИФИКАЦИИ

Интуитивно один из самых простых способов решить задачу классификации — провести границу между объектами разных классов (см. рисунок 1-2).

Задача классификации может быть:

- бинарной (два класса);
- многоклассовой (три и более классов).

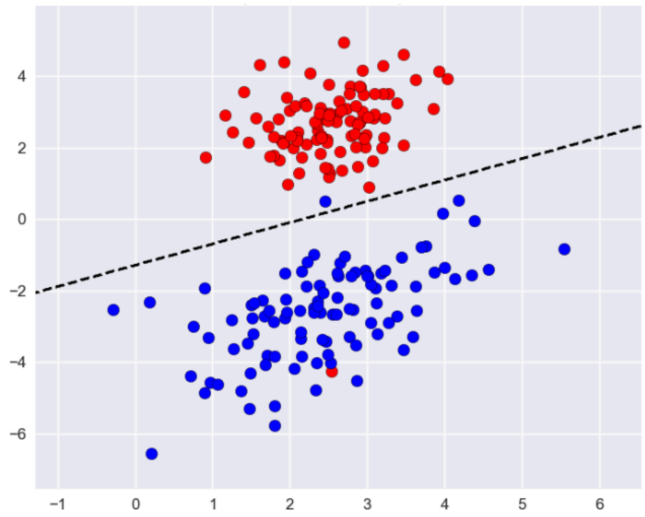


Рис. 1. Бинарная классификация.

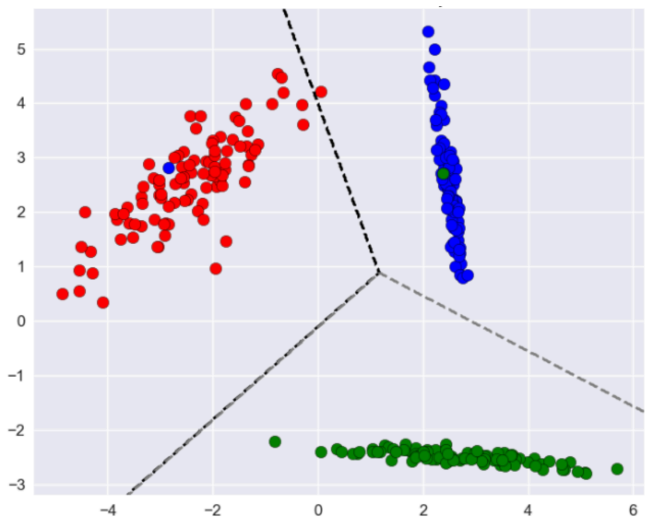


Рис. 2. Многоклассовая классификация.

Для бинарной классификации принято считать, что целевая переменная y принимает два возможных значения. Наиболее удобная для математики форма записи:

$$y \in \{-1, +1\},$$

где если y равен $+1$, то объект принадлежит положительному классу (например, «одобрить»); если y равен -1, то объект принадлежит отрицательному классу (например, «отказать»).

Такое обозначение удобно при математических выкладках, особенно в функциях потерь и при оптимизации.

Если классов больше двух, то мы говорим о многоклассовой классификации. В этом случае целевая переменная принимает значения от 1 до k, где k — количество классов:

$$y \in \{1,\ 2,\ \ldots,\ k\}.$$

При этом каждый объект принадлежит ровно одному из классов: в этом случае классы не пересекаются. Например, классификация видов цветов, классификация заболеваний, распознавание рукописных цифр от 0 до 9.

Основной подход к многоклассовой классификации — это использование моделей, которые сразу поддерживают несколько классов, например логистической регрессии с softmax. Но в рамках текущего раздела мы ограничимся только бинарной классификацией: рассмотрим ситуации, где нужно выбрать между двумя вариантами.

Это не упрощение — наоборот, большинство классических методов, включая логистическую регрессию и метод опорных векторов (SVM), изначально были разработаны именно для бинарных задач. Разобравшись, как они работают, будет проще понять, как на их основе решаются и более сложные многоклассовые задачи, о которых речь пойдёт далее в курсе.

Итак, задача бинарной классификации — это фактически задача построения разделяющей наши классы гиперплоскости. Иногда данные устроены «хорошо»: можно провести прямую (в двумерном случае, см. рисунок 3А) или гиперплоскость (в многомерном пространстве признаков), которая полностью разделяет объекты двух классов. К сожалению, в реальной жизни данные редко так идеальны. Часто классы перепутаны, пересекаются, содержат выбросы или даже значительно перекрываются. В этом случае никакая линейная граница не сможет полностью отделить один класс от другого без ошибок. Это ситуация нелинейной разделимости (см. рисунок 3Б).

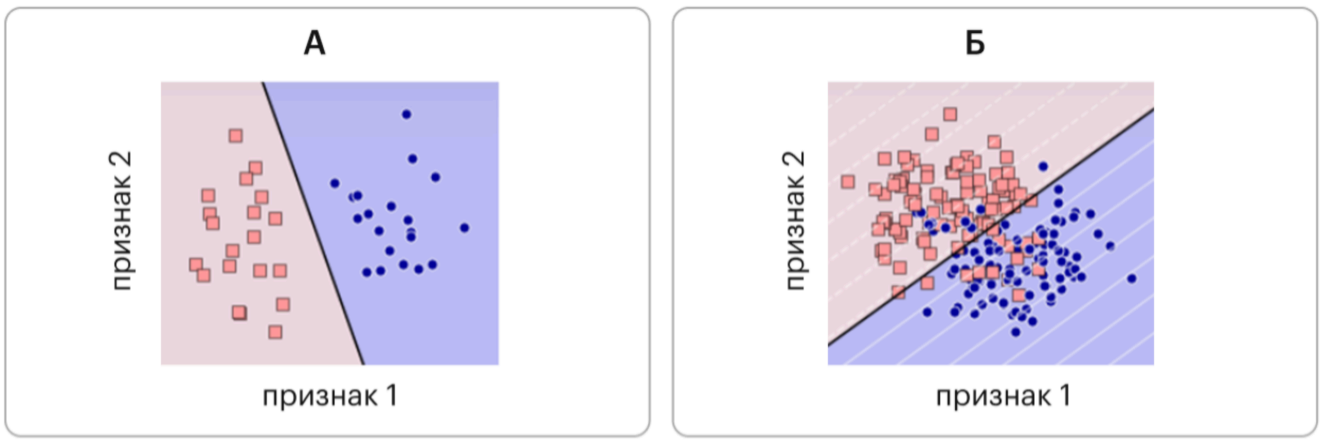


Рис. 3. Линейная и нелинейная разделимость классов. (А) Классы линейно разделимы (идеальная гиперплоскость). (Б) Классы нелинейно разделимы (перекрытие, выбросы).

Таким образом, перед нами встают два ключевых вопроса.

1. Как искать разделяющую прямую, если данные линейно разделимы?
2. Как действовать, когда без ошибок обойтись невозможно?

Далее в лонгриде мы разберём математическую постановку задачи построения разделяющей гиперплоскости. И в частности рассмотрим два наиболее популярных подхода для этого — логистическая регрессия и SVM.

1.2. ОСНОВНЫЕ МАТЕМАТИЧЕСКИЕ КОНЦЕПЦИИ ЗАДАЧИ КЛАССИФИКАЦИИ

Рассмотрим, как именно модель понимает, где провести такую линию между классами.

ОПРЕДЕЛЕНИЕ

Линейный классификатор — это модель, которая принимает решение о принадлежности объекта к тому или иному классу, сравнивая линейную комбинацию его признаков с нулевым порогом. Если значение линейной комбинации больше нуля, объект относят к одному классу, если меньше — к другому. То есть решение основано на знаке комбинации: $a(\mathbf{x}) = \text{sign}(\langle \mathbf{w}, \mathbf{x} \rangle + b)$.

ОПРЕДЕЛЕНИЕ

Линейная комбинация признаков — это выражение, в котором каждое значение каждого признака объекта умножается на некоторый коэффициент (вес), затем все эти произведения складываются. Например, если у нас есть признаки $x_1, x_2, \dots, x_n$ и соответствующие им веса $w_1, w_2, \dots, w_n$, то линейная комбинация выглядит так: $w_1 x_1 + w_2 x_2 + \cdots + w_n x_n$ — или в компактной форме как скалярное произведение: $\langle \mathbf{w}, \mathbf{x} \rangle$.

Формально линейный классификатор определяет класс объекта $\mathbf{x} \in \mathbb{R}^n$ по следующей формуле: $a(\mathbf{x}) = \text{sign}(\langle \mathbf{w}, \mathbf{x} \rangle + b)$.

- **w** — вектор весов; определяет направление разделяющей границы и то, насколько резко модель переходит от одного класса к другому; чем больше длина вектора $\|\mathbf{w}\|$, тем более резкий переход между классами.
- b — сдвиг или смещение; сдвигает гиперплоскость параллельно самой себе; если $b > 0$, граница смещается в сторону класса -1, если $b < 0$, то к классу $+1$.
- $\langle \mathbf{w}, \mathbf{x} \rangle$ — скалярное произведение; оно равно длине проекции вектора **x** на направление **w**, умноженной на длину **w** (или просто длине проекции, если **w** — единичный вектор).
  Знак этой величины показывает, по какую сторону от разделяющей гиперплоскости находится объект (см. рисунок 4, где стороны подписаны синим цветом):
  - $\langle \mathbf{w}, \mathbf{x} \rangle > 0$ → объект в направлении **w** (сторона класса $+1$);
  - $\langle \mathbf{w}, \mathbf{x} \rangle < 0$ → объект в противоположном направлении (сторона класса -1).
- $\text{sign}(\cdot)$ — знаковая функция; она возвращает:
  - $+1$, если выражение положительно;
  - -1, если отрицательно.

Таким образом, модель возвращает:

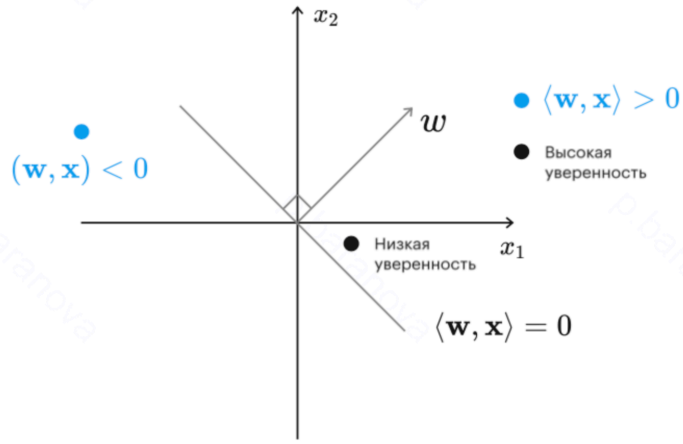
$$a(\mathbf{x}) = \begin{cases} +1, & \text{если } \langle \mathbf{w}, \mathbf{x} \rangle + b > 0, \\ -1, & \text{если } \langle \mathbf{w}, \mathbf{x} \rangle + b < 0. \end{cases}$$

Это и есть простейший линейный классификатор: он «разделяет» пространство признаков на две области, в одной из которых объекты относятся к классу $+1$, в другой — к классу -1.

Посмотрим, как эта модель работает геометрически. В пространстве признаков (допустим, двумерном) выражение $\langle \mathbf{w}, \mathbf{x} \rangle + b = 0$ задаёт гиперплоскость: в двумерном случае это прямая, в трёхмерном — плоскость, а в более высоких размерностях — гиперплоскость. Эта гиперплоскость и есть граница, разделяющая классы. Она «делит» всё пространство на две половины:

- с одной стороны — точки, где $\langle \mathbf{w}, \mathbf{x} \rangle + b > 0$ (класс $+1$);
- с другой — те, где $\langle \mathbf{w}, \mathbf{x} \rangle + b < 0$ (класс -1).

На рисунке 4 изображена реализация линейного классификатора в двумерном пространстве.



$$a(x) = \begin{cases} +1, & \text{если } \langle \mathbf{w}, \mathbf{x} \rangle + b > 0, \\ -1, & \text{если } \langle \mathbf{w}, \mathbf{x} \rangle + b < 0. \end{cases}$$

Рис. 4. Линейный классификатор в двумерном пространстве.

- Оси x_1 и x_2 — признаки объекта (например, доход и возраст клиента).
- Гиперплоскость $\langle \mathbf{w}, \mathbf{x} \rangle + b = 0$ — граница решений.
- Вектор нормали $\mathbf{w}$ перпендикулярен гиперплоскости, его направление задаёт, «куда смотрит» класс $+1$.
- Точки $\langle \mathbf{w}, \mathbf{x} \rangle + b > 0$ соответствуют предсказанному классу $+1$.
- Точки $\langle \mathbf{w}, \mathbf{x} \rangle + b < 0$ соответствуют предсказанному классу -1.

На рисунке 4 точки синего цвета показывают, как модель «оценивает» свою уверенность. Это связано с тем, как далеко точка находится от разделяющей гиперплоскости — чем дальше точка от границы, тем больше уверенность модели в своём решении.

Расстояние от точки $\mathbf{x}$ до гиперплоскости $\langle \mathbf{w}, \mathbf{x} \rangle + b = 0$ вычисляется по формуле:

$$\text{Расстояние} = \frac{|\langle \mathbf{w}, \mathbf{x} \rangle + b|}{\|\mathbf{w}\|},$$

где $\|\mathbf{w}\|$ — это норма вектора $\mathbf{w}$, его длина, которая вычисляется как $\|\mathbf{w}\| = \sqrt{w_1^2 + w_2^2 + \cdots + w_n^2}$. Допустим:

- точка A находится далеко от границы → большое расстояние → модель уверена: «Это точно класс $+1$»;
- точка B расположена близко к гиперплоскости → малое расстояние → модель сомневается: «Может, это класс $+1$?»

Однако в задачах классификации нас интересует не просто расстояние, а ещё и сторона, на которой находится точка. Важно учитывать знак выражения $\langle \mathbf{w}, \mathbf{x} \rangle + b$, ведь он показывает, в какой половине пространства находится объект.

Чтобы объединить расстояние до границы и принадлежность к классу,
вводится важное понятие — отступ.

1.2.1. Отступ

ОПРЕДЕЛЕНИЕ

Отступ (англ. margin) — это величина, равная произведению истинной метки класса $y \in \{-1, +1\}$ и значения линейной функции: $M(\mathbf{x}, y) = y \cdot (\langle \mathbf{w}, \mathbf{x} \rangle + b)$.

Отступ учитывает не только расстояние до разделяющей гиперплоскости, но и то, насколько правильно модель классифицирует объект (см. рисунок 5):

- если знак правильный и объект далеко от границы, отступ большой и положительный;
- если модель ошибается, отступ будет отрицательным;
- если модель почти не уверена, отступ близок к нулю.

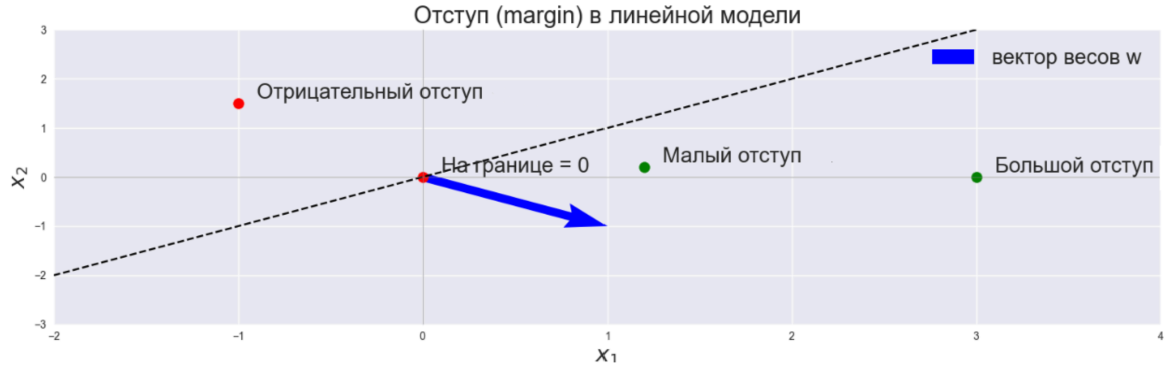


Рис. 5. Отступ в линейной модели.

Чтобы модель реже ошибалась, её нужно учить минимизировать количество ошибок классификации.
Для этого в обучении используют функции потерь.

1.3. ФУНКЦИИ ПОТЕРЬ

Если модель классифицировала объект правильно, то отступ $M_i \geq 0$; если ошиблась, $M_i \leq 0$.
Таким образом, мы можем посчитать количество ошибок так.

$$Q(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^{n} [M_i \leq 0] = \frac{1}{n} \sum_{i=1}^{n} [y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \leq 0],$$

где $[\cdot]$ — индикаторная функция, возвращает 1, если условие внутри выполняется, и 0, если иначе;
$y_i \in \{-1, +1\}$ — истинная метка объекта; $\langle \mathbf{w}, \mathbf{x}_i \rangle + b$ — «сырая» уверенность модели в принадлежности к классу $+1$.

Каждое слагаемое $[y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \leq 0]$ равно 1, если модель ошибается на объекте i, и 0, если предсказание верное.

Таким образом, сумма $\sum_{i=1}^{n} [y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \leq 0]$ подсчитывает количество ошибок на обучающей выборке, а деление на n даёт долю ошибок.

Однако в таком виде функция потерь не может быть использована для обучения. Есть две причины.

1. Это кусочно-постоянная функция, а следовательно: не дифференцируема, производная равна нулю почти везде, в точке разрыва не определена. Следовательно, мы не можем использовать градиентные методы, которые требуют хотя бы непрерывности, а желательно — гладкости (существования производной везде).
2. Функция не чувствительна к степени ошибки. Объект с $M_i = -0{,}1$ (почти у границы) и $M_i = -10$ (явный выброс) получают одинаковый штраф 1. Но интуитивно вторая ошибка «грубее», и модель должна это учитывать.

На рисунке ниже представлена данная проблема. На одной функции ошибки обозначены два неправильно классифицированных элемента. Один объект сильно ошибочно классифицирован, а второй — совсем чуть-чуть, но функция потерь этого не учитывает. Также у данной функции градиент всюду равен нулю кроме одной точки, где градиент — бесконечность.

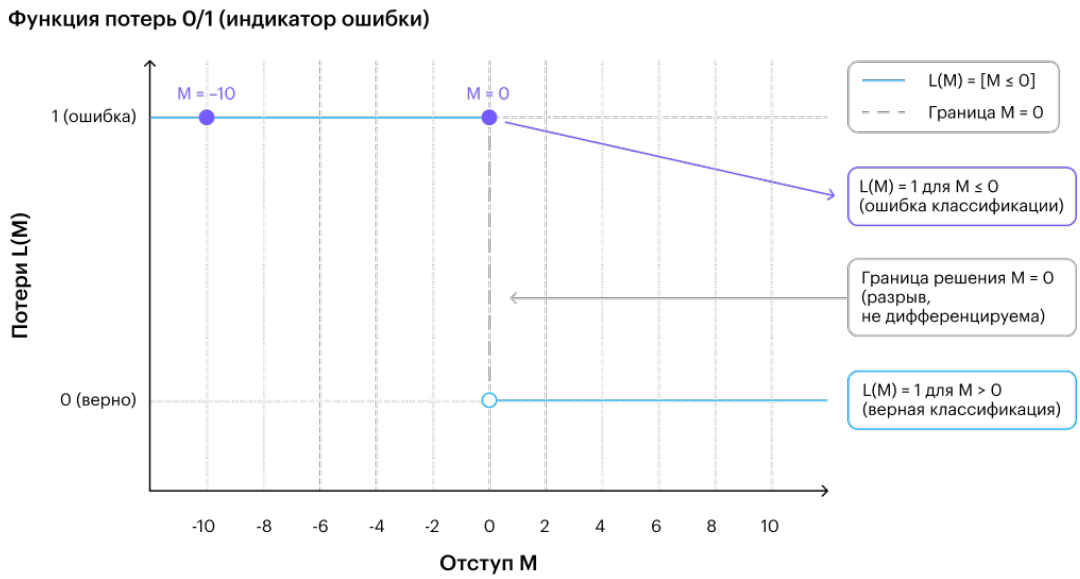


Рис. 6. Индикаторная функция потерь и её недостатки.

Идея — заменить «ступеньку» $M_i \leq 0$ на функцию, которая проще в оптимизации. Это классический приём в математике и машинном обучении: когда исходная функция неудобна (например, не дифференцируема), её заменяют на аппроксимацию — более «мягкую» функцию, которая ведёт себя похожим образом, но при этом позволяет использовать градиентные методы. В нашем случае нужно выбрать такую замену,

- которая имеет производные везде — гладкая;
- штрафует за отрицательные отступы — чем меньше M, тем больше штраф;
- сохраняет смысл — чем лучше разделение, тем меньше значение функционала.

Примеры таких функций приведены ниже.

Название	Формула	Краткая суть
Hinge loss	$L(M) = \max(0, 1 - M)$	Приводит к SVM
Logistic loss	$L(M) = \log(1 + e^{-M})$	Приводит к логистической регрессии
Exponential loss	$L(M) = e^{-M}$	Очень сильно штрафует за малый отступ, имеет экспоненциальный рост штрафа за ошибку. Используется в AdaBoost, где важно сильно выделить сложные объекты для последующих итераций
ArcTan loss	$\frac{2}{\pi}\arctan(-M) + 1$	Гладкая и ограниченная сверху и снизу: не даёт бесконечных штрафов. Хороша, когда нужно мягко подавлять ошибки, обеспечивает устойчивость к выбросам
Squared hinge loss	$L(M) = (\max(0, 1 - M))^2$	Модификация hinge loss с квадратичным штрафом. Сильнее штрафует за ошибки, но сохраняет те же свойства в плане классификации, как и обычный hinge
Huber loss	$L(M) = \begin{cases} \frac{1}{2}M^2, & \text{если } \lvert M\rvert \le 1 \\ \lvert M\rvert - \frac{1}{2}, & \text{если } \lvert M\rvert > 1 \end{cases}$	Компромисс между MSE (среднеквадратичной ошибкой) и абсолютной ошибкой. Сильно штрафует большие ошибки, но менее чувствительна к выбросам

Графическое сравнение описанных функций представлено на рисунке 7.

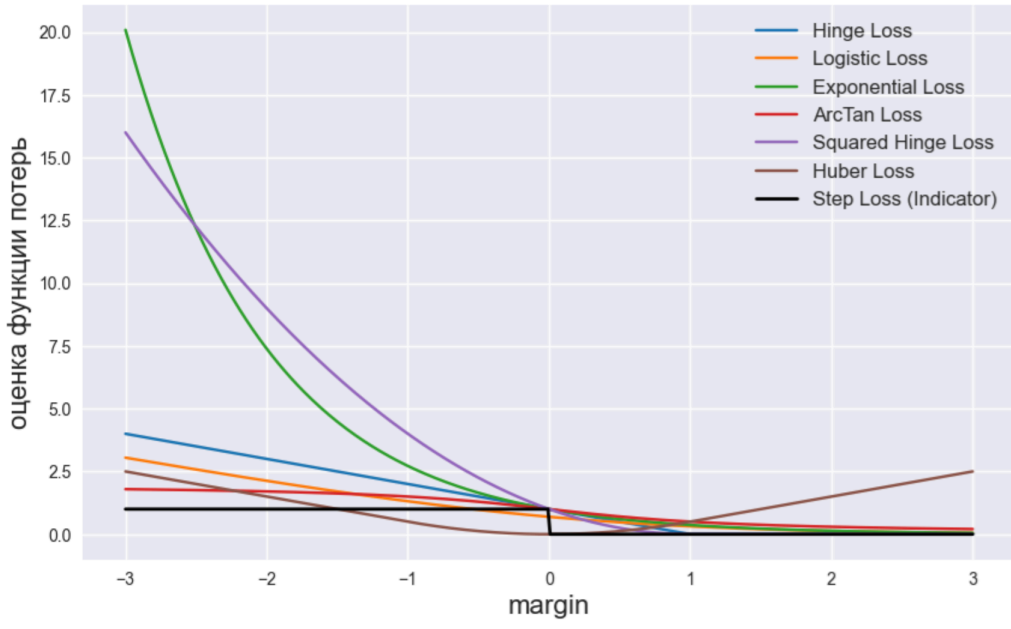


Рис. 7. Функции потерь.

В Python данные методы можно вызвать из sklearn следующим образом.

```python
# 1. Hinge loss
from sklearn.svm import LinearSVC

model = LinearSVC(loss = 'hinge', C = 1.0)
# C – контролирует силу регуляризации (будет рассмотрено далее)

# 2. Logistic loss
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(penalty = 'l2', C = 1.0, solver = 'lbfgs')
# C – сила регуляризации
# penalty – тип регуляризации: l1, l2, elasticnet
# solver – алгоритм оптимизации

# 3. Exponential loss
from sklearn.ensemble import AdaBoostClassifier

model = AdaBoostClassifier(n_estimators = 50, learning_rate = 1.0)
# n_estimators – число слабых классификаторов (далее в курсе про это будет)
# learning_rate – скорость обучения

# 4. Squared hinge loss
from sklearn.svm import LinearSVC

model = LinearSVC(loss = 'squared_hinge', C = 1.0)

# 5. Huber loss
from sklearn.linear_model import HuberRegressor

model = HuberRegressor()

# 6. ArcTanlLoss – реализации в sklearn нет
```

Как видно по блоку кода и таблице выше, человечество придумало много способов сглаживания кусочно-постоянных функций потерь. Далее в лонгриде мы подробно разберём два основных подхода для построения разделяющих границ — логистическую регрессию и метод опорных векторов (SVM).

Глава 2. Логистическая регрессия

До этого момента рассматривались модели, которые выдавали число, показывающее, к классу $+1$ или -1 ближе объект. Кредитная модель рассмотрела заявку клиента и вернула значение $1{,}8$. Непонятно, что это значит: одобрить заявку или нет. А если другому клиенту модель вернула $0{,}9$ — значит ли это, что он в два раза менее надёжен. Такие значения плохо масштабируются и не дают интуитивного понимания.

Логистическая регрессия позволяет превратить уверенность модели в практически значимую вероятность в интервале $[0,\ 1]$.

ОПРЕДЕЛЕНИЕ

Логистическая регрессия — это модель классификации, которая предсказывает вероятность принадлежности объекта к положительному классу, преобразуя линейную комбинацию признаков в значение от 0 до 1 с помощью сигмоиды. Параметры модели в классической постановке находят путём минимизации логистической функции потерь.

Результат работы модели логистической регрессией выражается формулой:

$$a(\mathbf{x}) = \mathbb{P}(y = +1 \mid \mathbf{x}),$$

где y — целевая переменная (класс); $\mathbf{x}$ — признаки наблюдения.

На картинке ниже производится сравнение результата работы линейной и логистической регрессии. Как ты помнишь, линейная регрессия предсказывает числа на непрерывном интервале. В целом можно обучить линейную регрессию предсказывать два различных класса и модель действительно будет предсказывать по большей части числа в этом диапазоне, но также она может предсказывать за пределами этого диапазона, что не согласуется с задачей (можно обрезать), а также могут возникнуть проблемы с оптимизацией, так как при такой постановке оптимизация будет вестись на MSE, а MSE от числа меньше 1 даёт маленькие градиенты.

Чтобы избавиться от данных проблем, было предложено нелинейно ограничивать пространство возможных ответов модели, что показано на рисунке справа.

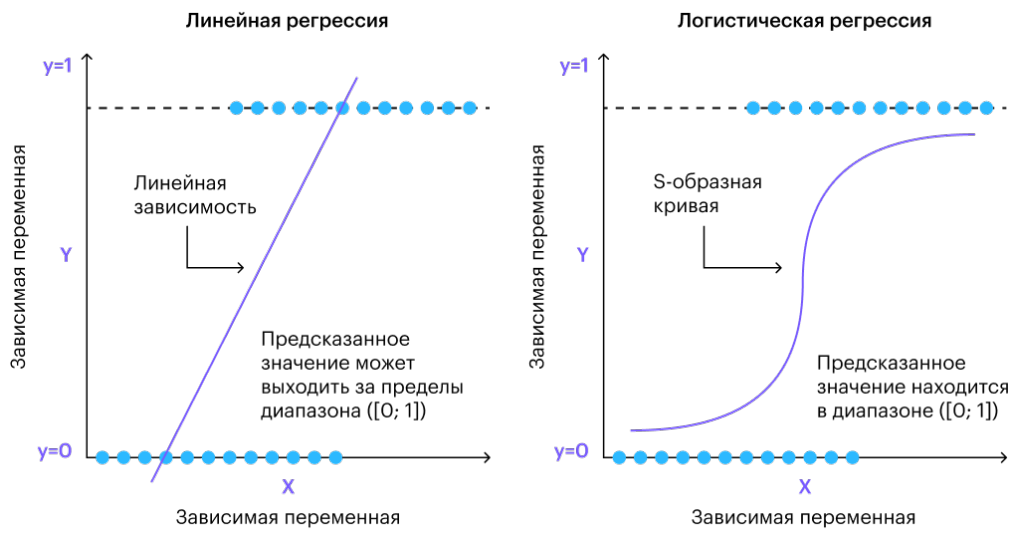


Рис. 8. Сравнение линейной и логистической регрессии. Источник: medium.com

2.1. ФУНКЦИЯ СИГМОИДЫ

Далее формально разберёмся, как логистическая регрессия превращает линейную комбинацию признаков в вероятность. Начнём с фундамента — сигмоидной функции (см. рисунок 9):

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

Сигмоида удобна тем, что она всегда возвращает значения в интервале от 0 до 1, независимо от того, насколько велико z.

- Если z очень большое, $\sigma(z)$ стремится к 1.
- Если z очень малое или отрицательное, $\sigma(z)$ стремится к 0.

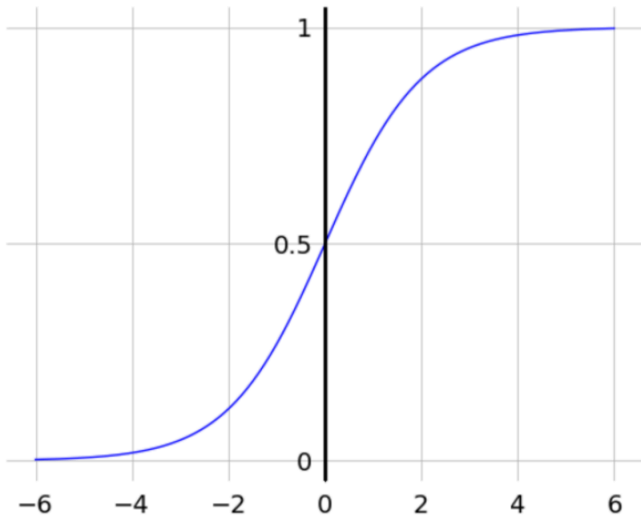


Рис. 9. График сигмоидной функции.

2.2. ПОЧЕМУ ИМЕННО СИГМОИДА?

Данную функцию взяли для преобразования пространства ответов в интервал $[0, 1]$ не просто так, а потому что пытались моделировать логарифм шансов (во сколько раз вероятность положительного класса больше вероятности отрицательного):

$$\text{odds} = \frac{P(y = +1 \mid \mathbf{x})}{P(y = -1 \mid \mathbf{x})} = \frac{p}{1 - p}.$$

Логарифмирование превращает шансы в неограниченную величину, которая может принимать любые значения от $-\infty$ до ∞. Раз шкала бесконечная, то будем её аппроксимировать моделью, способной на это. То есть линейной регрессией:

$$\log\left(\frac{p}{1 - p}\right) = \langle \mathbf{w}, \mathbf{x} \rangle.$$

Сделаем несложные математические преобразования и получим уравнение логистической регрессии:

$$\frac{p}{1 - p} = e^{\langle \mathbf{w}, \mathbf{x} \rangle} \quad \Longrightarrow \quad p = \frac{e^{\langle \mathbf{w}, \mathbf{x} \rangle}}{1 + e^{\langle \mathbf{w}, \mathbf{x} \rangle}} = \frac{1}{1 + e^{-\langle \mathbf{w}, \mathbf{x} \rangle}}.$$

2.3. ВЕРОЯТНОСТНАЯ ИНТЕРПРЕТАЦИЯ ЛОГИСТИЧЕСКОЙ РЕГРЕССИИ

Опираясь на этот вывод, мы можем записать вероятность положительного класса так:

$$p(y = +1 \mid \mathbf{x}) = \sigma(\langle \mathbf{w}, \mathbf{x} \rangle) = \frac{1}{1 + e^{-\langle \mathbf{w}, \mathbf{x} \rangle}},$$

где $\langle \mathbf{w}, \mathbf{x} \rangle$ — скалярное произведение весов модели **w** и признаков **x**.

Тогда вероятность отрицательного класса равна:

$$p(y = -1 \mid \mathbf{x}) = 1 - \sigma(\langle \mathbf{w}, \mathbf{x} \rangle).$$

Так как мы полностью задали параметрическую вероятностную модель, остаётся лишь найти такие параметры **w**, при которых наблюдаемые данные были бы наиболее правдоподобны. Для этого используется **метод максимального правдоподобия** (MLE). В случае бинарной классификации с сигмоидой он приводит к выпуклой задаче, которую можно эффективно решать градиентными методами (различные методы доступны в sklearn).

Кроме того, MLE даёт асимптотически несмещённую оценку вероятности: при большом объёме выборки и правильной спецификации модели выход логистической регрессии действительно сходится к истинной условной вероятности $P(y = +1 \mid \mathbf{x})$.

Подробнее мы это разберём в ближайших снизу разделах.

2.4. ЦЕЛЬ ОБУЧЕНИЯ: МАКСИМИЗАЦИЯ ПРАВДОПОДОБИЯ

ОПРЕДЕЛЕНИЕ

Правдоподобие (likelihood) — это функция параметров модели, которая показывает, насколько «вероятны» наблюдаемые данные при этих параметрах. Проще говоря, мы спрашиваем: «Если модель с заданными весами **w** действительно верна, с какой вероятностью мы получили бы именно такие метки y_i, какие видим в обучающей выборке?» Чем выше эта вероятность, тем лучше параметры модели объясняют данные.

ПРИМЕР

Пример. Есть три объекта, модель с текущими параметрами предсказывает для них вероятности принадлежности к положительному классу как $0{,}9$, $0{,}8$ и $0{,}7$. Если в реальности все три объекта действительно относятся к классу $+1$, то произведение этих вероятностей — это правдоподобие. В данном примере оно равно: $0{,}9 \times 0{,}8 \times 0{,}7 = 0{,}504$.

Цель обучения модели — найти такие **w**, чтобы это произведение было максимально большим. После изменения параметров предсказанные вероятности для объектов класса «+1» растут, например до $0{,}99$, $0{,}98$, $0{,}95$, тогда правдоподобие растёт до $0{,}922$, потому что такие предсказания лучше согласуются с реальностью.

Если хочется больше математической строгости для данного факта, то вот короткий, но качественный видеоразбор. Пусть у нас есть выборка из ℓ объектов с истинными метками y_i и предсказанными моделью вероятностями $\hat{y}_i$.

Функция правдоподобия для всей выборки будет произведением вероятностей для каждого наблюдения:

$$\mathcal{L}(\mathbf{w}) = \prod_{i=1}^{\ell} p(y_i \mid \mathbf{x}_i, \mathbf{w}),$$

где $p(y_i \mid \mathbf{x}_i, \mathbf{w})$ — вероятность того, что для объекта $\mathbf{x}_i$ с помощью модели с параметрами $\mathbf{w}$ был предсказан класс y_i.

Однако с произведением работать неудобно: вероятность для всех объектов в выборке может быть очень маленькой и при перемножении таких вероятностей мы просто уйдём за машинную точность в 0. Более того, логистическая регрессия обучается градиентными методами и вычислять градиент от произведения не очень удобно. Проще работать с логарифмом правдоподобия — он превращает произведение вероятностей в сумму элементов, существенно отличных от 0.

Логарифм правдоподобия для всей выборки имеет вид:

$$\log \mathcal{L}(\mathbf{w}) = \sum_{i=1}^{\ell} \log p(y_i \mid \mathbf{x}_i, \mathbf{w}).$$

Теперь, чтобы максимизировать правдоподобие, необходимо максимизировать сумму логарифмов вероятностей. Но поскольку мы часто решаем задачу минимизации функции потерь, то минимизируем отрицательный логарифм правдоподобия:

$$\mathcal{L}(\mathbf{w}) = -\sum_{i=1}^{\ell} \log p(y_i \mid \mathbf{x}_i, \mathbf{w}).$$

2.5. ЛОГАРИФМИЧЕСКАЯ ФУНКЦИЯ ПОТЕРЬ

Подставим вероятности, полученные из логистической регрессии, в логарифм правдоподобия. Итог — логарифмическая функция потерь для одного наблюдения:

$$\mathcal{L}(y_i, \hat{y}_i) = -\Big([y_i = +1] \log \sigma(\langle \mathbf{w}, \mathbf{x}_i \rangle) + [y_i = -1] \log \big(1 - \sigma(\langle \mathbf{w}, \mathbf{x}_i \rangle)\big)\Big).$$

Для всей выборки эта функция потерь будет выглядеть так:

$$-\sum_{i=1}^{\ell} \left([y_i = +1] \log \frac{1}{1 + \exp(-\langle \mathbf{w}, \mathbf{x}_i \rangle)} + [y_i = -1] \log \frac{\exp(-\langle \mathbf{w}, \mathbf{x}_i \rangle)}{1 + \exp(-\langle \mathbf{w}, \mathbf{x}_i \rangle)} \right) =$$

В дроби второго слагаемого домножаем числитель и знаменатель на $\exp(-\langle \mathbf{w}, \mathbf{x}_i \rangle)$

$$= -\sum_{i=1}^{\ell} \left([y_i = +1] \log \frac{1}{1 + \exp(-\langle \mathbf{w}, \mathbf{x}_i \rangle)} + [y_i = -1] \log \frac{1}{1 + \exp(\langle \mathbf{w}, \mathbf{x}_i \rangle)} \right) =$$

Расписываем отношение логарифма и раскрываем индикаторную функцию

$$= -\sum_{i=1}^{\ell} \left(-\log \left(1 + \exp(-\langle \mathbf{w}, \mathbf{x}_i \rangle)\right) - \log \left(1 + \exp(\langle \mathbf{w}, \mathbf{x}_i \rangle)\right) \right) =$$

Возвращаем таргет в формулу, чтобы схлопнуть два слагаемых

$$= \sum_{i=1}^{\ell} \log \left(1 + \exp(-y_i \langle \mathbf{w}, \mathbf{x}_i \rangle)\right) = \sum_{i=1}^{\ell} \log \left(1 + \exp(-M)\right).$$

Полученную функцию потерь минимизируют по параметрам **w**, чтобы модель с наибольшей вероятностью объясняла наблюдаемые данные. То есть мы строим с точки зрения функции потерь оптимальную разделяющую гиперплоскость, которая максимизирует зазор (margin) между классами, что отражено в итоговой формуле через обозначение M.

2.5.1. Связь с бинарной кросс-энтропией

Теперь покажем, что только что выведенная функция потерь для логистической регрессии на самом деле эквивалентна бинарной кросс-энтропии с первой недели курса.

Мы записывали бинарную кросс-энтропию как:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{\ell}\sum_{i=1}^{\ell}\Big[y_i \log p_i + (1-y_i)\log(1-p_i)\Big],$$

где $y_i \in \{0,1\}$ — истинная метка, p_i — предсказанная вероятность класса 1.

1. Подставляем p_i в кросс-энтропию.
   Для $y_i = 1$:

$$-\log p_i = -\log \frac{1}{1+\exp(-\langle \mathbf{w}, \mathbf{x}_i\rangle)} = \log(1+\exp(-\langle \mathbf{w}, \mathbf{x}_i\rangle)).$$

   Для $y_i = 0$:

$$-\log(1-p_i) = -\log \frac{\exp(-\langle \mathbf{w}, \mathbf{x}_i\rangle)}{1+\exp(-\langle \mathbf{w}, \mathbf{x}_i\rangle)} = \log(1+\exp(\langle \mathbf{w}, \mathbf{x}_i\rangle)).$$

2. Собираем вместе.

$$\mathcal{L}_{\text{CE}} = \frac{1}{\ell}\left[\sum_{i:y_i=1}\log(1+\exp(-\langle \mathbf{w}, \mathbf{x}_i\rangle)) + \sum_{i:y_i=0}\log(1+\exp(\langle \mathbf{w}, \mathbf{x}_i\rangle))\right].$$

3. Переходим к меткам $y_i \in \{-1,+1\}$. Если обозначить $y_i = +1$ для класса 1 и $y_i = -1$ для класса 0, то два слагаемых объединяются в одно:

$$\mathcal{L}_{\log} = \frac{1}{\ell}\sum_{i=1}^{\ell}\log\left(1+\exp(-y_i\langle \mathbf{w}, \mathbf{x}_i\rangle)\right).$$

Таким образом, логистическая функция потерь — это в точности бинарная кросс-энтропия после подстановки сигмоидной вероятности и перехода к симметричным меткам $\{-1,+1\}$.

2.6. ИНТЕРПРЕТАЦИЯ ПРЕДСКАЗАННЫХ ВЕРОЯТНОСТЕЙ

Мы определили, что логистическая регрессия для каждого объекта **x** выдаёт число $a(\mathbf{x}) \in [0,1]$, которое интерпретируется как оценка вероятности принадлежности к положительному классу:

$$a(\mathbf{x}) \approx P(y=+1 \mid \mathbf{x}).$$

Модель строит аппроксимацию именно данной величины, так как это следует из функции потерь (лог-лосс), для которого оптимальным предсказанием является именно $P(y=+1|\mathbf{x})$.

Поэтому, если модель хорошо обучена и откалибрована (об этом чуть ниже в лонгриде), её выход естественно можно интерпретировать как вероятность, но что она значит на практике? Если модель предсказывает $a(\mathbf{x}) = 0{,}8$, это не гарантирует, что конкретный объект окажется положительным. Утверждение относится не к одному объекту, а к **группе** объектов, для которых модель выдала такую же (или близкую) вероятность. То есть такой ответ модели показывает, что среди большого числа таких объектов примерно 80% будут иметь истинную метку $+1$. Например, если взять 100 объектов с вероятностью $0{,}8$, около 80 из них действительно принадлежат классу $+1$.

Теперь обоснуем это формально. Для этого рассмотрим все объекты, получившие от модели одно и то же предсказание b. Всего таких предсказаний n.

Доля положительных объектов в этой группе может быть записана как:

$$\frac{1}{n}\sum_{i=1}^{n}\mathbb{I}[y_i = +1],$$

где $\mathbb{I}[\cdot]$ — индикатор (1, если условие выполнено, иначе 0).

По закону больших чисел, при росте n эта эмпирическая доля сходится к истинной условной вероятности:

$$\lim_{n\to\infty}\frac{1}{n}\sum_{i=1}^{n}\mathbb{I}[y_i = +1] = P(y = +1 \mid \mathbf{x}).$$

Таким образом, цепочка преобразований выглядит так:

$$\text{предсказание модели } b \;\approx\; \text{доля положительных среди объектов с } b \xrightarrow[n\to\infty]{} P(y = +1 \mid \mathbf{x}).$$

Следовательно, $a(\mathbf{x})$ действительно можно рассматривать как оценку вероятности $P(y = +1|\mathbf{x})$, так как логистическая регрессия строит оценку для истинной вероятности. **Что вероятность не означает:**

- вероятность $0{,}8$ **не говорит** «объект на 80% принадлежит классу $+1$». Истинная метка всегда либо $+1$, либо -1. Вероятность характеризует **уверенность модели**, а не частичную принадлежность;
- вероятность не заменяет решения о классе. Классификация получается после выбора порога (обычно $0{,}5$). Объекты с предсказаниями $0{,}51$ и $0{,}99$ попадут в класс $+1$, хотя степень уверенности модели в них разная.

Преимущества вероятностного ответа

1. **Интерпретируемость.** Вместо абстрактного числа модель выдаёт вероятность, которую можно напрямую использовать для принятия решений. Вероятность $0{,}8$ означает высокую уверенность в положительном классе, а $0{,}2$ — низкую.
2. **Настраиваемый порог.** По умолчанию порог бинаризации $0{,}5$, но его можно менять в зависимости от бизнес-требований. Если важнее минимизировать ложноположительные срабатывания (FP), порог повышают, например до $0{,}7$ или $0{,}8$. Если критичнее не пропустить положительный класс (минимизировать FN), порог снижают.

Для закрепления разберём финальный пример. Пусть модель выдала вероятность $0{,}8$ для десяти разных объектов. Если модель хорошо откалибрована, то в среднем **около 8** из них действительно принадлежат классу $+1$, а около 2 — классу -1. Именно этот смысл мы вкладываем в слова «вероятность положительного класса $0{,}8$».

2.7. ПРИМЕНЕНИЕ ЛОГИСТИЧЕСКОЙ РЕГРЕССИИ

Логистическая регрессия по-прежнему используется в бизнесе, например в банковском кредитном скоринге или при присвоении кредитных рейтингов, благодаря своей простоте, интерпретируемости и возможности гибко настраивать пороги для оценки рисков.

Наверняка ты знаешь, что это далеко не самая сильная модель, но тем не менее, она действительно используется для перечисленных выше задач, что закреплено на регуляторном уровне в рамках стандартов «Базель 2», «Базель 3» и МСФО 9. Регулятор обязывает использовать именно эту модель именно из-за простоты интерпретаций.

Также данная модель очень часто встречается в эконометрических исследованиях благодаря своей возможности получать крайне понятные человеку интерпретации. Разберём всё на примере задаче кредитного скоринга для заёмщика в банке. В банк поступает заявка на потребительный кредит от клиента со следующими характеристиками.

Возраст	**32 года**
Ежемесячный доход	85 000 ₽
Сумма кредита	600 000 ₽
Срок кредита	36 месяцев
Кредитный рейтинг	680
DTI (Debt to income) Соотношение долга и дохода	0,42
Просрочки в прошлом	1

Логистическая регрессия вычисляет линейную комбинацию признаков (после обработки взяли некоторые переменные):

$$DefaultRate = w_1 \cdot DTI + w_2 \cdot CreditScore + w_3 \cdot \ln_Income + w_4 \cdot PastDefaults + b.$$

$DefaultRate$ — вероятность дефолта, то есть наша целевая переменная.

Допустим, модель выучила соответствующие коэффициенты.

DTI	**+2,3**
CreditScore	$-0{,}003$
ln_Income	$-0{,}25$
PastDefaults	$+1{,}2$
Свободный член b	$-0{,}5$

Для них можно дать ряд содержательных интерпретаций.

- **DTI = +2,3**: чем выше долговая нагрузка заемщика относительно его дохода, тем выше риск дефолта. При увеличении DTI на 1 процентный пункт (например, с 40% до 41%) шансы дефолта возрастают примерно на $2{,}3\%$.
- **CreditScore =** $-0{,}003$: более высокий кредитный рейтинг снижает риск дефолта. Увеличение кредитного рейтинга на 10 пунктов уменьшает шансы дефолта примерно на 3%. Например, заемщик с рейтингом 700 считается менее рискованным, чем заемщик с рейтингом 690.
- **Логарифм дохода =** $-0{,}25$: более высокий доход связан со снижением риска дефолта. Если ежемесячный доход заёмщика увеличивается в два раза, шансы дефолта уменьшаются примерно на 16–18%, поскольку клиенту проще обслуживать свои обязательства.
- **Просрочки = +1,2**: каждая дополнительная просрочка в кредитной истории существенно увеличивает риск. Шансы дефолта возрастают примерно в $e^{1{,}2} \approx 3{,}3$ раза по сравнению с заёмщиком, имеющим на одну просрочку меньше.

Помимо содержательных интерпретаций, можно и нужно использовать модель по прямому назначению — предсказывать вероятность дефолта заемщика. Подставим значения нашего клиента:

$$z = 2{,}3 \cdot 0{,}42 + (-0{,}003) \cdot 680 + (-0{,}25) \cdot \ln(85000) + 1{,}2 \cdot 1 + (-0{,}5) = -3{,}211.$$

Применяем сигмоидную функцию:

$$p(\text{дефолт}) = \frac{1}{1 + e^{-(-3{,}211)}} \approx 0{,}039.$$

То есть клиент не вернёт кредит с вероятностью в $3{,}9\%$. Допустим, банк готов брать на себя риск в 20%, в таком случае клиенту кредит будет выдан.

Используя эту вероятность, банк может оценить стоимость риска, сколько надо денег заложить на резервы в случае дефолта клиента. Вычисляются они как произведение стоимость кредита и вероятности дефолта, то есть:

$$EL = 600\,000 \cdot 0{,}039 = 23\,400\ ₽.$$

2.8. ИТОГИ ЛОГИСТИЧЕСКОЙ РЕГРЕССИИ

- Логистическая регрессия — модель бинарной классификации, которая превращает линейную комбинацию признаков в вероятность положительного класса при помощи сигмоидной функции: $a(\mathbf{x}) = P(y = +1|\mathbf{x}) \in [0, 1]$.
- Выбор сигмоиды продиктован предположением о линейной зависимости логарифма шансов (log-odds) от признаков; это даёт прямую интерпретацию коэффициентов: каждый вес показывает вклад признака в изменение логарифма шансов.
- Обучение модели основано на максимизации правдоподобия (MLE), что эквивалентно минимизации логистической функции потерь (она же бинарная кросс-энтропия). Задача выпуклая и решается градиентными методами.
- Предсказанная вероятность допускает частотную интерпретацию: для хорошо откалиброванной модели среди объектов с одинаковым скором доля действительно положительных приблизительно равна этому скору.

- Модель проста, вычислительно эффективна и интерпретируема, поэтому широко применяется в кредитном скоринге, риск-менеджменте и других областях, где важна объяснимость и соблюдение регуляторных стандартов.
- Основное ограничение — линейная разделяющая граница; для задач со сложной структурой данных требуется конструирование дополнительных признаков или переход к более гибким моделям.
- Несмотря на свою простоту помимо требований регулятора, она находит применение в задачах, где данных очень мало или в качестве метаклассификатора, чтобы получить весовую схему для ответов других алгоритмов, которые решают задачу на исходных признаках.

Мы рассмотрели модель, которая умеет предсказывать вероятности. Позже разберём, насколько этим вероятностям можно доверять и как их улучшить. Поговорим об этом после знакомства со вторым классическим методом классификации — SVM.

Глава 3. SVM

ОПРЕДЕЛЕНИЕ

Support vector machine (SVM), или метод опорных векторов, — это алгоритм классификации, который ищет разделяющую гиперплоскость с максимальным отступом (margin). Отступ определяется как минимальное расстояние от гиперплоскости до ближайших объектов обучающей выборки.

Модель старается не просто правильно классифицировать объекты, но сделать это с максимальной «уверенностью» — чтобы все объекты лежали как можно дальше от границы разделения (см. рисунок 10Г).

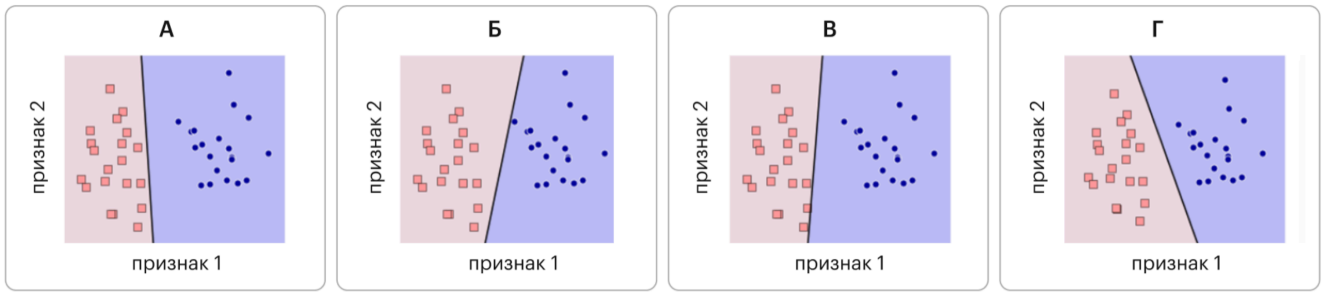


Рис. 10. Примеры проведения границы между двумя классами.

В случае А разделяющая граница смещена слишком близко к красным точкам — отступ до ближайших объектов этого класса минимален. В случае Б гиперплоскость хоть и проходит между классами, но делает это под неудачным наклоном, в результате чего расстояние до ближайших точек минимально. В случае В теперь граница смещена и к красному классу, но теперь в отличие от А ещё, кажется, под неудачным углом. В случае Г оптимальный вариант — граница проходит посередине и максимально удалена от ближайших точек обоих классов, что соответствует максимальному отступу. Все эти границы правильно разделяют наши объекты, но только Г кажется интуитивно правильной.

Поэтому в этом блоке разберём, каким образом получать оптимальную разделяющую поверхность и откуда взялись термины «вектор» и «опорные» в SVM.

3.1. ПОЧЕМУ «ВЕКТОРЫ»

Представим, что SVM — это архитектор, который строит стену между двумя группами людей в парке так,

- чтобы она разделяла группы абсолютно верно (никто не должен быть на «неправильной» стороне);
- стена была максимально удалена от обеих групп (если люди слегка сдвинутся, они не пересекут стену).

В контексте SVM «вектор» — это объект данных, представленный как точка в n-мерном пространстве признаков. Пример:

- если классифицировать фрукты по весу и цвету, каждый фрукт — это вектор вида (вес, цвет), где признаки представлены числами;
- если анализировать тексты, вектор может быть набором частот слов.

Гиперплоскость — это «стена», которая разделяет векторы (точки) разных классов. Математически гиперплоскость задаётся уравнением: $\langle \mathbf{w}, \mathbf{x} \rangle + b = 0$, где **w** — вектор весов (нормаль к гиперплоскости), **x** — вектор признаков объекта, b — смещение (bias).

Вектор весов **w** — это компас гиперплоскости, так как это направление, перпендикулярное гиперплоскости (её нормаль). Он определяет, как именно «повёрнута» разделяющая граница. Параметр b определяет, насколько гиперплоскость смещена вдоль направления вектора **w**.

SVM ищет гиперплоскость так, чтобы (в линейно разделимом случае):

- все точки класса $+1$ удовлетворяли условию $\langle \mathbf{w}, \mathbf{x} \rangle + b \geq 1$;
- все точки класса -1 удовлетворяли условию $\langle \mathbf{w}, \mathbf{x} \rangle + b \leq -1$.

Число 1 здесь выбрано для удобства: оно задаёт нормировку, при которой опорные векторы лежат на границе «полосы» шириной 2.

3.2. ПОЧЕМУ ВЕКТОРЫ «ОПОРНЫЕ»

Опорные векторы — это точки следующего вида:

- $\langle \mathbf{w}, \mathbf{x} \rangle + b = 1$ (для класса $+1$);
- $\langle \mathbf{w}, \mathbf{x} \rangle + b = -1$ (для класса -1).

Если убрать все точки, кроме опорных векторов, гиперплоскость останется той же — они как бы держат границы отступа, как колонны держат крышу (см. рисунок 11).

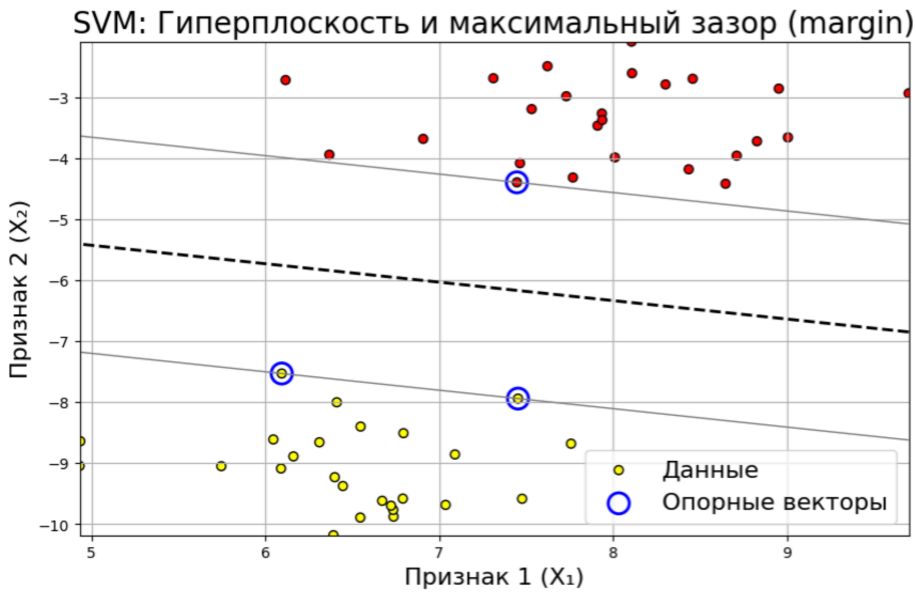


Рис. 11. Опорные векторы.

Рассмотрим, как математически выводится формула максимизации отступа для линейно разделимого случая.

3.3. ВЫВОД ФОРМУЛЫ ОТСТУПА ДЛЯ SVM

Ранее мы ввели понятие отступа для произвольного линейного классификатора:

$$M(\mathbf{x}, y) = y \cdot (\langle \mathbf{w}, \mathbf{x} \rangle + b).$$

Если отступ положителен — объект классифицирован правильно, если отрицателен — модель ошибается. Теперь задача — построить **оптимальную** разделяющую гиперплоскость, которая не просто разделяет классы, но делает это наилучшим образом. Под наилучшим образом мы понимаем такую гиперплоскость, середина которой максимально удалена от точек противоположных классов. То есть, если смотреть на рисунок 12, то для нас предпочтительнее гиперплоскость A, так как она обладает существенно большей шириной полосы (расстоянием между классами), чем гиперплоскость B.

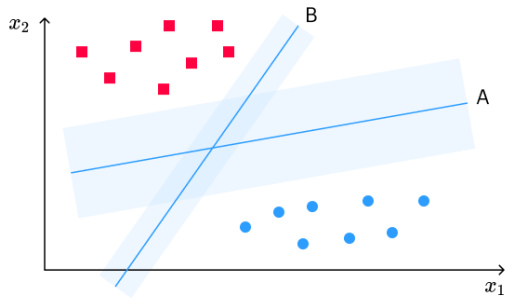


Рис. 12. Варианты разделяющей гиперплоскости. Источник: proproprogs.ru

Перейдём к математической постановке поиска такой гиперплоскости. Для любой гиперплоскости $\langle \mathbf{w}, \mathbf{x} \rangle + b = 0$ расстояние от точки $\mathbf{x}$ до неё равно:

$$\text{Расстояние} = \frac{|\langle \mathbf{w}, \mathbf{x} \rangle + b|}{\|\mathbf{w}\|}.$$

Но нас интересует не просто расстояние, а расстояние **до ближайшей точки каждого класса** с учётом правильности классификации. Поэтому вводят **геометрический отступ** (margin) для объекта $(\mathbf{x}_i, y_i)$:

$$\gamma_i = \frac{y_i \cdot (\langle \mathbf{w}, \mathbf{x}_i \rangle + b)}{\|\mathbf{w}\|}.$$

Напомним, что ранее мы определили **функциональный отступ** как $\hat{\gamma}_i = y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b)$. Легко заметить, что геометрический отступ получается из функционального делением на норму вектора весов:

$$\gamma_i = \frac{\hat{\gamma}_i}{\|\mathbf{w}\|}.$$

Геометрический отступ γ_i положителен, если объект находится с правильной стороны от гиперплоскости, и его абсолютная величина равна расстоянию от точки до гиперплоскости. Если же объект классифицирован неверно, $\gamma_i < 0$, модуль говорит, насколько глубоко точка «вторглась» на чужую территорию.

Идея SVM: выбрать такие $\mathbf{w}$ и b, при которых **минимальный геометрический отступ** по всем объектам обучающей выборки будет максимален. То есть мы хотим, чтобы гиперплоскость проходила как можно дальше от ближайших точек обоих классов.

Обозначим минимальный геометрический отступ за γ:

$$\gamma = \min_i \frac{y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b)}{\|\mathbf{w}\|}.$$

Требуется максимизировать γ по $\mathbf{w}$, b. То есть мы выбираем объекты, которые лежат «на краю» каждого класса и пытаемся расположить между ними разделяющую поверхность таким образом, чтобы её ширина была максимальна.

Однако в такой постановке есть неоднозначность: гиперплоскость $\langle \mathbf{w}, \mathbf{x} \rangle + b$ и гиперплоскость $\langle c\mathbf{w}, \mathbf{x} \rangle + cb$ при $c > 0$ — одна и та же геометрическая поверхность. Масштабируя параметры, можно неограниченно увеличивать числитель в выражении для γ, не меняя саму гиперплоскость.

3.4. НОРМИРОВКА И ОПОРНЫЕ ВЕКТОРЫ

Чтобы убрать этот произвол, фиксируется масштаб, накладывая условия на **опорные векторы** — точки, ближайшие к разделяющей гиперплоскости. А именно, для этих точек (или хотя бы для тех, что лежат на границе зазора) полагают:

$$y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) = 1.$$

Тогда **функциональный отступ** для опорных векторов становится равным 1, а соответствующий **геометрический отступ** —

$$\gamma = \frac{1}{\|\mathbf{w}\|}.$$

Для всех остальных точек тогда автоматически должно быть:

$$y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \geq 1,$$

поскольку они находятся не ближе, чем опорные точки. Таким образом, две границы, отделяющие «полосу безопасности» вокруг гиперплоскости, задаются уравнениями:

$$\langle \mathbf{w}, \mathbf{x} \rangle + b = +1 \quad \text{(граница класса } +1\text{)}; \qquad \langle \mathbf{w}, \mathbf{x} \rangle + b = -1 \quad \text{(граница класса } -1\text{)}.$$

Эти две параллельные гиперплоскости отстоят от разделяющей гиперплоскости на одинаковое расстояние по разные стороны. Сама разделяющая гиперплоскость $\langle \mathbf{w}, \mathbf{x} \rangle + b = 0$ находится строго посередине между ними.

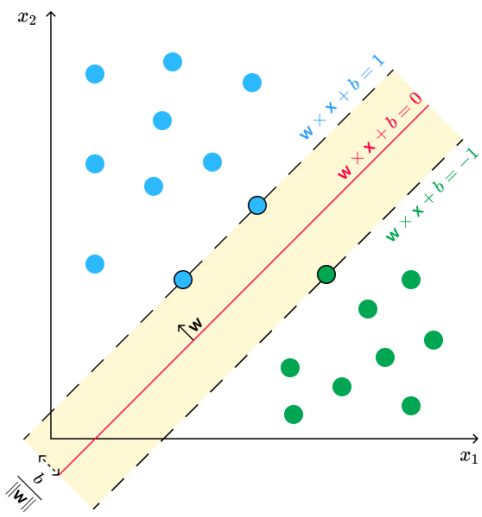


Рис. 13. Визуализация «полосы безопасности» между точками и гиперплоскостью. Источник: proproprogs.ru

Теперь вычислим ширину полосы, разделяющей классы. Возьмём две точки $\mathbf{x}_+$ и $\mathbf{x}_-$, лежащие на границах $\langle \mathbf{w}, \mathbf{x}_+ \rangle + b = 1$ и $\langle \mathbf{w}, \mathbf{x}_- \rangle + b = -1$ соответственно, такие, что прямая, соединяющая их, перпендикулярна обеим границам (чёрные отрезки на рисунке ниже).

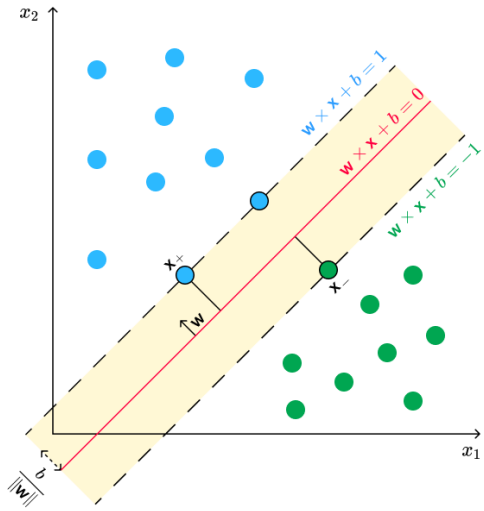


Рис. 14. Добавляем расстояние от опорных точек до гиперплоскости. Источник: proproprogs.ru

Тогда ширина зазора — это расстояние между этими двумя точками. Его можно найти, вычтя одну точку из другой:

$$\langle \mathbf{w}, \mathbf{x}_+ \rangle - \langle \mathbf{w}, \mathbf{x}_- \rangle = 1 - (-1); \qquad \langle \mathbf{w}, \mathbf{x}_+ - \mathbf{x}_- \rangle = 2.$$

Поскольку $\mathbf{x}_+ - \mathbf{x}_-$ направлен вдоль $\mathbf{w}$, его можно записать как $\lambda \mathbf{w}$ для некоторого скаляра $\lambda > 0$. Тогда:

$$\langle \mathbf{w}, \lambda \mathbf{w} \rangle = 2; \qquad \lambda \|\mathbf{w}\|^2 = 2 \quad \Rightarrow \quad \lambda = \frac{2}{\|\mathbf{w}\|^2}.$$

Ширина полосы — это длина вектора $\mathbf{x}_+ - \mathbf{x}_-$:

$$\|\mathbf{x}_+ - \mathbf{x}_-\| = \|\lambda \mathbf{w}\| = \lambda \|\mathbf{w}\| = \frac{2}{\|\mathbf{w}\|^2} \|\mathbf{w}\| = \frac{2}{\|\mathbf{w}\|}.$$

Таким образом, **ширина отступа** (margin width) между ближайшими точками противоположных классов равна $\frac{2}{\|\mathbf{w}\|}$.

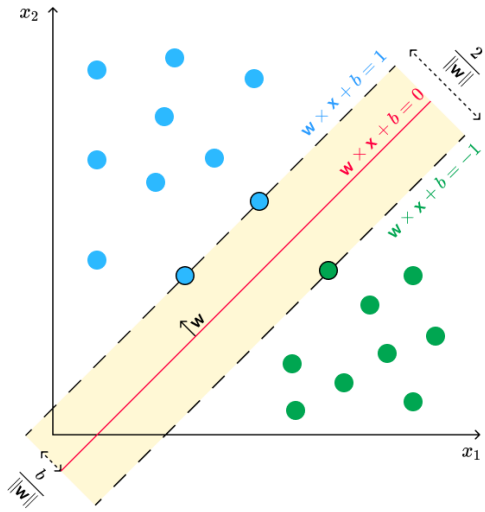


Рис. 15. Финальная разделяющая гиперплоскость. Источник: proproprogs.ru

3.5. ЗАДАЧА ОПТИМИЗАЦИИ ДЛЯ ЛИНЕЙНО РАЗДЕЛИМОГО СЛУЧАЯ

Итак, мы хотим:

- максимизировать ширину зазора $\frac{2}{\|\mathbf{w}\|}$ (2 геометрических отступа γ), что соответствует минимизации $\frac{1}{2}\|\mathbf{w}\|^2$ и одновременно делает задачу оптимизации гладкой и выпуклой;
- чтобы каждый объект лежал не ближе, чем на расстоянии $\frac{1}{\|\mathbf{w}\|}$ от разделяющей гиперплоскости, и при этом находился с правильной стороны.

Тогда в предположении, что данные линейно разделимы без ошибок, задача SVM с жёстким зазором записывается так:

$$\begin{cases} \frac{1}{2}\|\mathbf{w}\|^2 \to \min\limits_{\mathbf{w},b}, \\ y_i\left(\langle \mathbf{w}, \mathbf{x}_i \rangle + b\right) \geq 1, \quad i = 1, \dots, \ell. \end{cases}$$

Решение этой задачи даёт **оптимальную разделяющую гиперплоскость**, которая определяется только опорными векторами — теми точками, для которых ограничение выполняется как равенство.

3.6. SVM ДЛЯ ЛИНЕЙНО НЕРАЗДЕЛИМЫХ ДАННЫХ

Реальные данные редко бывают идеально линейно разделимыми. Даже если разделимость есть, жёсткое следование условию $y_i\left(\langle \mathbf{w}, \mathbf{x}_i \rangle + b\right) \geq 1$ может привести к переобучению из-за выбросов.

Поэтому вводят **мягкий зазор** (soft margin), разрешая некоторым точкам нарушать границу отступа. Для этого используются **переменные ослабления** $\xi_i \geq 0$. Условие для каждого объекта смягчается.

$$y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad i = 1, \dots, \ell.$$

Интуиция:

- если $\xi_i = 0$, точка лежит за границей margin (идеальный случай);
- если $0 < \xi_i < 1$, точка внутри margin, но классифицируется верно;
- если $\xi_i \geq 1$, точка попадает на «чужую» сторону (ошибка классификации).

Создадим баланс между двумя целями:

- максимизация ширины отступа $\frac{2}{\|\mathbf{w}\|}$;
- минимизация суммарного штрафа $C \sum\limits_{i=1}^{\ell} \xi_i$.

Параметр $C > 0$ управляет компромиссом между шириной зазора и допустимостью нарушений.

- Маленькое C — модель «прощает» много ошибок, отдавая приоритет широкому зазору; приводит к более простой, но менее точной границе (возможно недообучение).
- Большое C — модель стремится минимизировать нарушения, зазор может сужаться, граница подстраивается под выбросы (риск переобучения).
- При $C \to \infty$ soft-margin переходит в hard-margin (если данные разделимы).

Этот компромисс оформляется в виде новой задачи оптимизации:

$$\begin{cases} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum\limits_{i=1}^{\ell}\xi_i \to \min\limits_{\mathbf{w},b,\xi}, \\ y_i(\langle \mathbf{w}, \mathbf{x}_i\rangle + b) \geq 1 - \xi_i, \\ \xi_i \geq 0, \quad i = 1, \ldots, \ell. \end{cases}$$

Задача остаётся выпуклой, что гарантирует единственное решение. Условия на ξ_i можно переписать в явном виде. Из второго и третьего ограничений видно, что каждая ξ_i должна быть не меньше нуля и не меньше величины $1 - y_i(\langle \mathbf{w}, \mathbf{x}_i\rangle + b)$. Это значит, $\xi_i \geq \max(0,\ 1 - y_i(\langle \mathbf{w}, \mathbf{x}_i\rangle + b))$.

Поскольку ξ_i входят в целевую функцию с положительными коэффициентами ($C \geq 0$) и подлежат минимизации, в оптимальном решении каждое ξ_i принимает минимально возможное значение, допустимое ограничениями. Тогда в точке оптимума обязательно выполняется равенство:

$$\xi_i = \max(0,\ 1 - y_i(\langle \mathbf{w}, \mathbf{x}_i\rangle + b)).$$

На рисунке ниже представлено сравнение с hard margin. Как уже было сказано, данный подход просто допускает наличие наблюдений, которые «вторгаются» на территорию другого класса или лежат на расстоянии менее 1 от разделяющей гиперплоскости.

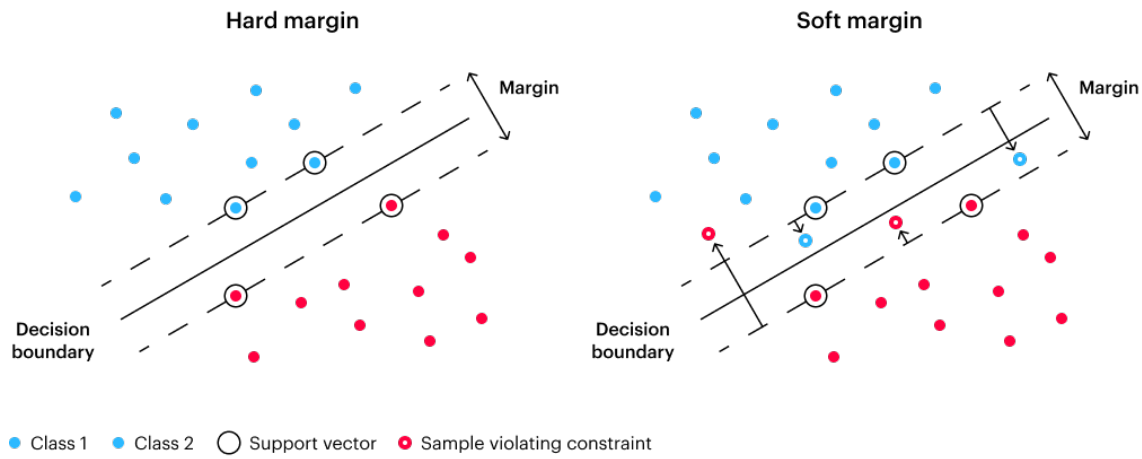


Рис. 16. Сравнение hard и soft margin SVM. Источник: axelmendoza.com

Равенство $\xi_i = \max(0,\ 1 - y_i(\langle \mathbf{w}, \mathbf{x}_i\rangle + b))$ позволяет исключить переменные ξ_i и соответствующие ограничения, подставив выражение для ξ_i в целевую функцию. То есть возможно просто выписать лагранжиан. В результате получаем эквивалентную задачу безусловной оптимизации, зависящую только от параметров модели $\mathbf{w}$ и b:

$$\frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{\ell}\max(0,\ 1 - y_i(\langle \mathbf{w}, \mathbf{x}_i\rangle + b)) \to \min_{\mathbf{w},b},$$

где $\frac{1}{2}\|\mathbf{w}\|^2$ — регуляризация, предотвращающая переобучение;

$C\sum\limits_{i=1}^{\ell}\max(0,\ 1 - y_i(\langle \mathbf{w}, \mathbf{x}_i\rangle + b))$ — hinge loss, штрафует за ошибки и точки внутри margin.

Далее задача может решаться градиентными методами.

Глава 4. Ядерный трюк (kernel trick)

До сих пор мы предполагали, что данные можно разделить линейной границей. Но на практике это условие выполняется далеко не всегда: во многих задачах объекты разных классов оказываются перемешаны так, что провести между ними прямую уже невозможно. В этом разделе рассмотрим, как метод опорных векторов справляется с такими ситуациями с помощью ядерного трюка.

ОПРЕДЕЛЕНИЕ

Ядерный трюк (kernel trick) — это математический приём, позволяющий алгоритмам работать в пространствах высокой размерности, не вычисляя новые признаки явно. Вместо этого алгоритм вычисляет лишь попарные скалярные произведения между объектами с помощью специальных функций — *ядер*.

Это решает проблему линейно неразделимых данных: мы можем мысленно отобразить их в пространство большей размерности, где они станут линейно разделимыми, и при этом избежать колоссальных вычислительных затрат, просто подставив ядро вместо обычного скалярного произведения.

ПРИМЕР

Пример. В исходном 2D-пространстве два класса — вложенные окружности (как в датасете `make_circles`), не разделимые прямой. Но если добавить новое измерение $z = x_1^2 + x_2^2$, то в 3D-пространстве классы могут оказаться разделимыми плоскостью (см. рисунок 17).

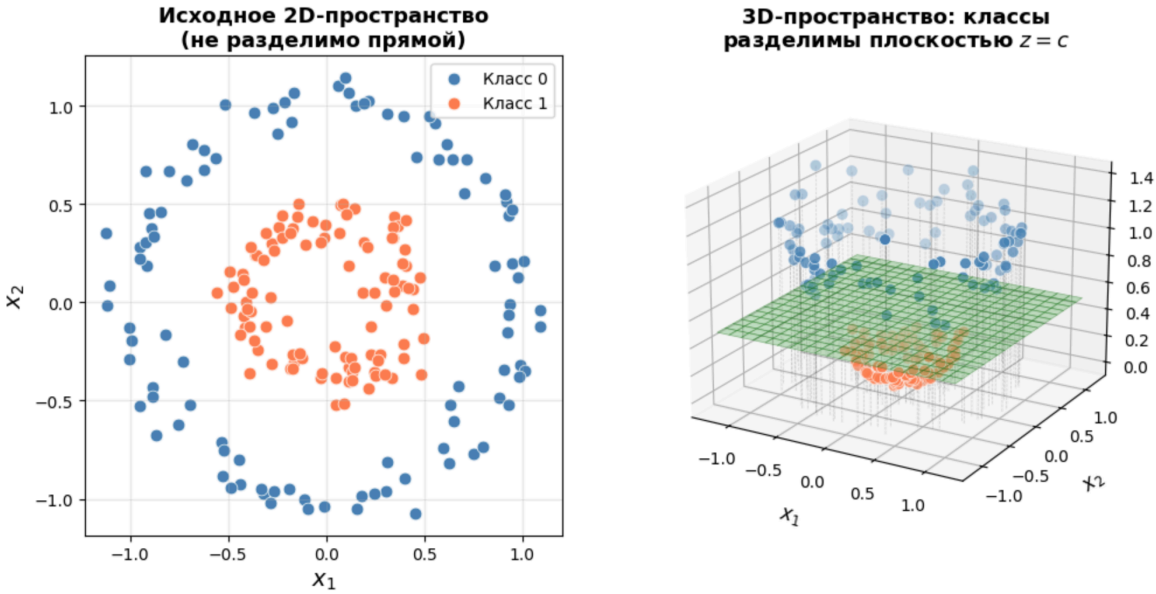


Рис. 17. Пример построения разделяющей гиперплоскости в новом признаковом пространстве.

ОПРЕДЕЛЕНИЕ

Признаковое пространство — это пространство, где каждый объект описывается своими признаками, записанными в виде вектора чисел.

Однако напрямую вычислять отображение в высокоразмерное пространство затратно или даже невозможно. Вместо этого используют ядерный трюк.

Идея: подставлять функцию $K(\mathbf{x}_i, \mathbf{x}_j)$, которая ведёт себя так, будто мы работаем в пространстве высокой размерности — это и есть ядерный трюк.

Чтобы понять концепцию ядерного трюка, необходимо перейти от исходной задачи SVM к двойственной. Этот переход раскрывает фундаментальное свойство SVM: его зависимость исключительно от попарных скалярных произведений объектов.

4.1. ИСХОДНАЯ ЗАДАЧА SVM

Исходная формулировка SVM минимизирует норму вектора весов (максимизирует отступ) с учётом ошибок классификации через hinge loss:

$$\frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{\ell}\max(0, 1 - y_i \cdot f(\mathbf{x}_i)) \to \min_{\mathbf{w},b},$$

где $f(\mathbf{x}_i) = \langle \mathbf{w}, \mathbf{x}_i \rangle + b$.

4.2. ПЕРЕХОД К ДВОЙСТВЕННОЙ ЗАДАЧЕ

1. Заменяем hinge loss явными переменными ошибок ξ_i для аналитического удобства:

$$\begin{cases} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{\ell}\xi_i \to \min_{\mathbf{w},b,\xi}, \\ y_i(\langle \mathbf{w}, \mathbf{x}_i \rangle + b) \geq 1 - \xi_i, \quad \xi_i \geq 0. \end{cases}$$

   Переменные ξ_i позволяют объектам нарушать границу отступа, что принципиально для работы с нелинейно разделимыми данными.

2. Вводим множители Лагранжа:
   - $\alpha_i \geq 0$ для ограничений отступа;
   - $\mu_i \geq 0$ для ограничений $\xi_i \geq 0$.

   Лагранжиан принимает вид

$$\mathcal{L} = \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{\ell}\xi_i - \sum_{i=1}^{\ell}\alpha_i\left[y_i\left(\langle \mathbf{w}, \mathbf{x}_i \rangle + b\right) - 1 + \xi_i\right] - \sum_{i=1}^{\ell}\mu_i\xi_i.$$

3. Условия оптимальности. Приравниваем производные к нулю.
   - По $\mathbf{w}$: $\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \mathbf{w} - \sum_{i=1}^{\ell}\alpha_i y_i \mathbf{x}_i = 0 \quad \Longrightarrow \quad \mathbf{w} = \sum_{i=1}^{\ell}\alpha_i y_i \mathbf{x}_i.$
   - По b: $\frac{\partial \mathcal{L}}{\partial b} = -\sum_{i=1}^{\ell}\alpha_i y_i = 0 \quad \Longrightarrow \quad \sum_{i=1}^{\ell}\alpha_i y_i = 0.$
   - По ξ_i: $\frac{\partial \mathcal{L}}{\partial \xi_i} = C - \alpha_i - \mu_i = 0 \quad \Longrightarrow \quad \alpha_i = C - \mu_i$. Так как $\mu_i \geq 0$, то $0 \leq \alpha_i \leq C$.

4. Формирование двойственной задачи. Подставляем $\mathbf{w} = \sum_{i=1}^{\ell}\alpha_i y_i \mathbf{x}_i$ в лагранжиан. Раскрывая $\|\mathbf{w}\|^2$, получаем

$$\|\mathbf{w}\|^2 = \left\langle \sum_{i=1}^{\ell}\alpha_i y_i \mathbf{x}_i, \ \sum_{j=1}^{\ell}\alpha_j y_j \mathbf{x}_j \right\rangle = \sum_{i=1}^{\ell}\sum_{j=1}^{\ell}\alpha_i \alpha_j y_i y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle.$$

После упрощения двойственная задача принимает вид

$$\begin{cases} \sum\limits_{i=1}^{\ell} \alpha_i \;-\; \frac{1}{2} \sum\limits_{i=1}^{\ell} \sum\limits_{j=1}^{\ell} \alpha_i \alpha_j y_i y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle \;\to\; \max, \\ \sum\limits_{i=1}^{\ell} \alpha_i y_i = 0, \\ 0 \le \alpha_i \le C. \end{cases}$$

Оптимизация зависит только от попарных скалярных произведений $\langle \mathbf{x}_i, \mathbf{x}_j \rangle$ между объектами обучающей выборки. Исходные признаки $\mathbf{x}_i$ больше не требуются — достаточно меры их «сходства».

4.2.1. Предсказания

Используя оптимальный вектор весов $\mathbf{w}^* = \sum\limits_{i=1}^{\ell} \alpha_i^* \, y_i \, \mathbf{x}_i$, получим прогноз для нового объекта:

$$f(\mathbf{x}_{\text{new}}) = \langle \mathbf{w}^*, \mathbf{x}_{\text{new}} \rangle + b^* = \sum_{i=1}^{\ell} \alpha_i^* \, y_i \, \langle \mathbf{x}_i, \mathbf{x}_{\text{new}} \rangle + b^*.$$

Формула предсказания также зависит от скалярных произведений, но теперь — от произведений между обучающими объектами и новым объектом.

4.2.2. Ядерный трюк

Предположим, данные нелинейно разделимы в исходном пространстве. Мы хотим преобразовать их в пространство более высокой размерности через $\phi(\mathbf{x})$, где линейная разделимость возможна: $\phi : \mathbf{x} \mapsto \phi(\mathbf{x})$.

Тогда скалярные произведения в формулах обучения и предсказания заменяются на скалярные произведения в преобразованном пространстве признаков:

$$\langle \mathbf{x}_i, \mathbf{x}_j \rangle \to \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle.$$

Но напрямую вычислять $\phi(\mathbf{x})$ часто невозможно (оно может быть бесконечномерным). Поэтому вводят ядерную функцию:

$$K(\mathbf{x}_i, \mathbf{x}_j) \;:=\; \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle.$$

Интуиция ядра:

- ядро $K(\mathbf{x}, \mathbf{z})$ измеряет «сходство» объектов в скрытом пространстве признаков;
- если подобрать подходящее ядро, SVM сможет строить нелинейные границы в исходном пространстве, оставаясь линейным в скрытом пространстве.

Таким образом:

- в двойственной задаче: $\sum\limits_{i=1}^{\ell} \alpha_i \;-\; \frac{1}{2} \sum\limits_{i=1}^{\ell} \sum\limits_{j=1}^{\ell} \alpha_i \alpha_j y_i y_j \, K(\mathbf{x}_i, \mathbf{x}_j) \;\to\; \max$;

- в предсказании: $f(\mathbf{x}_{\text{new}}) = \sum_{i=1}^{\ell} \alpha_i \, y_i \, K(\mathbf{x}_i, \mathbf{x}_{\text{new}}) + b.$

Таким образом, ключевая особенность SVM в двойственной форме: и оптимизация, и предсказание зависят не от индивидуальных значений признаков, а исключительно от попарных скалярных произведений $\langle \mathbf{x}_i, \mathbf{x}_j \rangle$ (или их ядерных аналогов $K(\mathbf{x}_i, \mathbf{x}_j)$). Именно это позволяет применять ядерный трюк — не вычисляя громоздкое отображение ϕ, достаточно задать функцию сходства K, и SVM будет работать в соответствующем скрытом пространстве.

4.3. ЯДЕРНЫЕ ФУНКЦИИ

Разные ядерные функции соответствуют разным способам отображения данных в признаковое пространство. От выбора ядра зависит форма разделяющей поверхности. Рассмотрим несколько популярных примеров.

4.3.1. Полиномиальное ядро (polynomial kernel)

Использует полиномиальные функции для отображения данных. Подходит для задач, где граница между классами имеет полиномиальную форму (кольца, спирали).

Формула:

$$K(\mathbf{x}_i, \mathbf{x}_j) = (\langle \mathbf{x}_i, \mathbf{x}_j \rangle + c)^d,$$

где $\mathbf{x}_i, \mathbf{x}_j \in \mathbb{R}^n$ — два вектора признаков;

$c \geq 0$ — смещение;

$d \in \mathbb{N}$ — степень полинома.

Интерпретация: это ядро соответствует отображению в пространство всех мономов степени до d от исходных признаков. Например, если $\mathbf{x} = (x_1, x_2)$ и $d = 2$, то отображение $\phi(\mathbf{x})$ будет включать: 1, x_1, x_2, x_1^2, x_2^2, $x_1 x_2$ — 6 признаков.

В scikit-learn вызывается следующей функцией.

```
SVC(kernel='poly', degree=3, coef0=1),
# degree – степень полинома d
# coef0 – смещение c
```



Рис. 18. Пример использования полиномиального ядра на синтетически сгенерированных данных.

4.3.2. Гауссово (RBF) ядро

Использует радиальную базисную функцию, подходит для задач с неочевидной структурой (например, кластеры сложной формы).

Формула:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2),$$

где γ — гиперпараметр, чем он больше, тем сложнее граница (возрастает риск переобучения).

Интерпретация:

- это ядро соответствует отображению в бесконечномерное пространство признаков;
- оно даёт высокое значение только для близких точек — в таком пространстве объекты группируются в локальные «пузырьки» (отсюда и «радикальные», или «радиальные», базисы).

В scikit-learn вызывается следующей функцией.

```
SVC(kernel='rbf', gamma='scale')
# gamma соответствует параметру γ
```

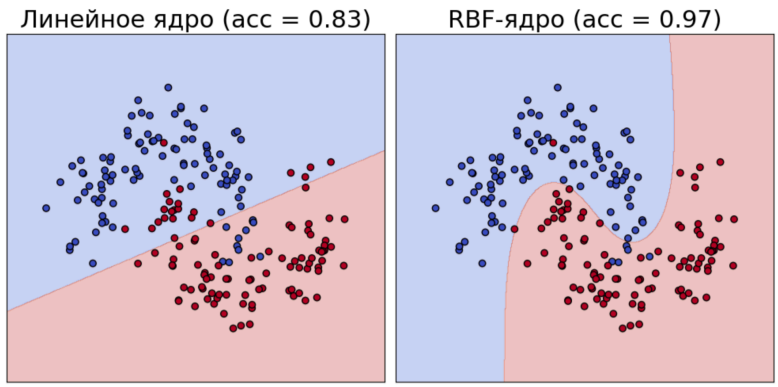


Рис. 19. Пример использования гауссова ядра на синтетически сгенерированных данных.

Глава 5. Практическое применение и сравнение моделей

Мы разобрали два популярных алгоритма линейной классификации и увидели, что оба они успешно решают схожие задачи. Теперь сравним их между собой и разберёмся, в каких ситуациях каждый из них оказывается предпочтительнее.

5.1. ПРИМЕНЕНИЕ SVM

Сейчас алгоритм SVM применяется в практике гораздо реже, чем раньше, из-за своей вычислительной сложности по размеру выборки $O(n^2)$. В 1990–2000-х он считался одним из самых перспективных методов машинного обучения и показывал отличные результаты на небольших и средних данных.

Со временем появились более эффективные алгоритмы, например градиентный бустинг (XGBoost, LightGBM) и нейронные сети, которые лучше справляются с большими данными и сложными задачами. О бустингах пойдёт речь далее в нашем курсе.

SVM до сих пор используется в задачах с небольшими объёмами данных и высокими требованиями к интерпретируемости разделяющей гиперплоскости, которая имеет геометрический смысл. Но главное — SVM помогает лучше понять фундаментальные идеи: границы решений, максимальный отступ (margin), регуляризацию, ядровые методы. Эти концепции лежат в основе многих современных алгоритмов:

- Ядра используются в методах kernel trick (например, kernel PCA, kernel ridge regression).
- Идея максимизация отступа вдохновила подходы к регуляризации и margin-based learning (например, в бустинге и некоторых нейронных сетях).

5.2. ОТЛИЧИЯ SVM И ЛОГИСТИЧЕСКОЙ РЕГРЕССИИ

И логистическая регрессия, и SVM используются для бинарной классификации, но они по-разному обращаются с отступом (margin).

- Логистическая регрессия стремится уводить отступы в бесконечность — это позволяет ей оценивать вероятности принадлежности к классам.
- SVM требует лишь, чтобы отступ был больше единицы. Всё, что сверх этого, уже не влияет на значение функции потерь.

На рисунке 12 это различие хорошо видно: при отрицательных значениях отступа (где модель ошибается) сигмоидальная и hinge-функции ведут себя похоже — обе резко возрастают, штрафуя модель. Но при положительных отступах поведение расходится: функция потерь логистической регрессии продолжает плавно убывать, стремясь к нулю, в то время как hinge-функция просто становится линейной до точки 1, а затем обрывается — она считает дальнейший рост отступа неважным.

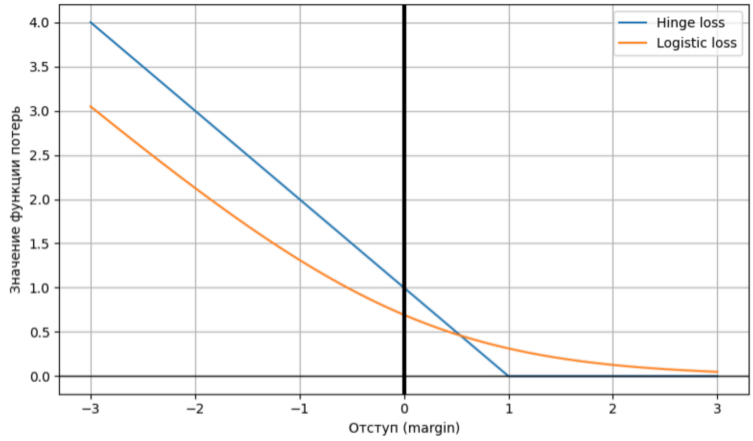


Рис. 20. Сравнение Hinge Loss и Logistic Loss.

Однако логистическая регрессия предоставляет возможность интерпретируемости: она выдаёт вероятности, на основе которых можно гибко принимать решения. Именно поэтому она до сих пор широко применяется в бизнесе — особенно в задачах, где важно объяснять модель и принимать решения на основе рисков (обнаружение мошенничества, кредитный скоринг, прогноз заболевания).

Прочие сравнения можно найти в таблице ниже.

Критерий	SVM	LogReg
Выход модели	Расстояние до границы со знаком	Вероятность принадлежности к классу
Интерпретируемость	Геометрическая: опорные векторы и ширина зазора; веса указывают направление и важность признаков	Вероятностная: можно оценивать риски, настраивать порог принятия решений
Чувствительность к выбросам	При правильно подобранном C фокусируется на опорных векторах, менее чувствителен к далёким выбросам	Более чувствительна: каждый объект влияет на решение, выбросы могут сильно смещать границу
Работа с большими выборками	Вычислительно сложнее (особенно с нелинейными ядрами), плохо масштабируется на миллионы объектов	Быстрее обучается, хорошо масштабируется, имеет эффективные солверы для больших данных
Вероятностная калибровка	Не предусмотрена, требует дополнительной калибровки (об этом дальше в лонгриде)	Выход сразу интерпретируется как вероятность

Критерий	SVM	LogReg
Ядерный трюк	Естественно встраивается через двойственную задачу, позволяет строить нелинейные границы	Возможен (kernel logistic regression), но менее распространён и вычислительно затратен
Применимость	Малые и средние данные, высокая размерность, задачи, где важна сама разделяющая гиперплоскость (например, текст, биоинформатика)	Широко применяется в бизнесе: кредитный скоринг, маркетинг, медицина — везде, где нужна интерпретируемая вероятность

Когда что выбирать?

- Если критичны именно вероятности (например, для оценки рисков или расчёта ожидаемых потерь) — логистическая регрессия.
- Если важнее найти наиболее «уверенную» разделяющую полосу, данные невелики, и есть подозрение на сложную нелинейную структуру — SVM с подходящим ядром может дать более точную границу.
- На практике при больших объёмах данных и стандартных бизнес-задачах чаще применяют логистическую регрессию или градиентный бустинг, однако понимание SVM необходимо для освоения ядровых методов и идей максимизации отступа.

5.3. ПОЧЕМУ SVM НЕ ВЫДАЁТ ВЕРОЯТНОСТИ НАПРЯМУЮ

SVM оптимизирует только ширину зазора и штрафует за попадание объектов внутрь него или на чужую сторону. После того как объект оказывается на правильной стороне на расстоянии не менее $\frac{1}{\|\mathbf{w}\|}$ (то есть при отступе ≥ 1), его положение перестаёт влиять на функцию потерь — она становится равной нулю.

Модель «не интересует», насколько далеко уверенный объект находится от границы: её цель — найти саму границу и зазор, а не выстроить монотонную шкалу уверенности. Поэтому расстояние до гиперплоскости, которое SVM выдаёт для нового объекта, нельзя напрямую интерпретировать как степень принадлежности к классу или как вероятность. Оно отражает скорее структурный зазор, но не калибровано под частоту положительных исходов. Именно поэтому для получения вероятностей из SVM требуется дополнительная калибровка (например, метод Платта, которая обучается отдельно и подгоняет сигмоиду к выходам модели).

Глава 6. Калибровка вероятностей

Алгоритмы классификации выдают не только предсказания классов, но и вероятности. Однако эти вероятности часто бывают неоткалиброванными: модель может переоценивать или недооценивать уверенность в предсказаниях.

Во многих задачах недостаточно просто знать класс объекта — необходимо получить **корректную оценку вероятности** этого класса. Хорошо откалиброванные вероятности позволяют делать следующее.

- **Принимать решения на основе ожидаемой выгоды или потерь.** Если модель предсказывает вероятность дефолта заёмщика, банк умножает её на сумму кредита и оценивает ожидаемые потери. Неоткалиброванная вероятность в 90%, которая исторически соответствует реальной доле дефолтов лишь в 50%, приведёт к завышению резервов и неоправданному отказу в кредитах.
- **Сравнивать риски для разных объектов.** В медицине калиброванные вероятности позволяют корректно ранжировать пациентов по срочности вмешательства. Если модель даёт вероятность инфаркта 70% для пациента А и 30% для пациента Б, но на самом деле эти вероятности не соответствуют реальным частотам, порядок приоритетов может оказаться ошибочным.
- **Устанавливать осмысленные пороги для бинарных решений.** При калиброванных вероятностях порог $0{,}5$ действительно означает, что положительный исход случается в 50% случаев. Без калибровки порог может не иметь такой интерпретации.
- **Вычислять ожидаемые значения метрик, зависящих от вероятностей.** В рекламных системах ожидаемый доход вычисляется как predicted probability of click × стоимость клика. Смещённые вероятности напрямую искажают оценку эффективности кампании.

Таким образом, калибровка превращает выходы модели в **настоящие вероятности**, которые можно буквально интерпретировать как частоты событий. Именно это свойство критично во всех ситуациях, где на основе предсказаний принимаются финансовые, медицинские или операционные решения.

ПРИМЕР

Неоткалиброванная модель предсказала вероятность заболевания у 100 пациентов как 90%, но реально болеют только 50 человек — это переоценка вероятностей (90% против фактических 50%). После калибровки прогнозы могут скорректироваться до 55%, приблизившись к реальному распределению.

ОПРЕДЕЛЕНИЕ

Калибровка — это процесс приведения выходных вероятностей модели в соответствие с реальной вероятностью наступления события.

Анализ калибровки проводят, разбивая предсказания по значениям вероятностей на группы (бины) и сравнивая в каждой группе среднюю предсказанную вероятность с фактической долей положительных исходов.

Рассмотрим пример.

1. **Разбиваем весь диапазон предсказанных вероятностей на интервалы (бины).** Например, 10 бинов: $[0{,}0–0{,}1]$, $[0{,}1–0{,}2]$, … , $[0{,}9–1{,}0]$.
2. **Группируем объекты по бинам.** В бин $[0{,}7–0{,}8]$ попадают все объекты, для которых модель предсказала вероятность класса $+1$ в этом диапазоне (допустим, их оказалось 100).

3. **Вычисляем два числа для каждого бина:**
   - **Средняя предсказанная вероятность** $\overline{p}$ по всем объектам бина.
   - **Фактическая доля положительных примеров** y_{pos} в этом бине (сколько из них реально принадлежат классу $+1$). Например, если из 100 объектов действительно положительных 75, то фактическая доля равна $0{,}75$, а среднее предсказание может быть $0{,}77$.
4. **Сравниваем среднюю предсказанную вероятность с фактической долей.** Если $\overline{p} \approx y_{pos}$ (как $0{,}77$ и $0{,}75$), модель хорошо откалибрована в этом интервале. Если $\overline{p}$ систематически занижена или завышена относительно y_{pos}, модель нуждается в калибровке.
5. **Визуализируем сравнение с помощью калибровочной кривой.**
   - По оси X откладываются средние предсказанные вероятности для каждого бина.
   - По оси Y — фактическая доля положительных примеров в этих бинах. Идеально откалиброванная модель даёт кривую, совпадающую с прямой $y = x$.

Таким образом, калибровочная кривая наглядно показывает, насколько предсказанные вероятности соответствуют реальным частотам классов.

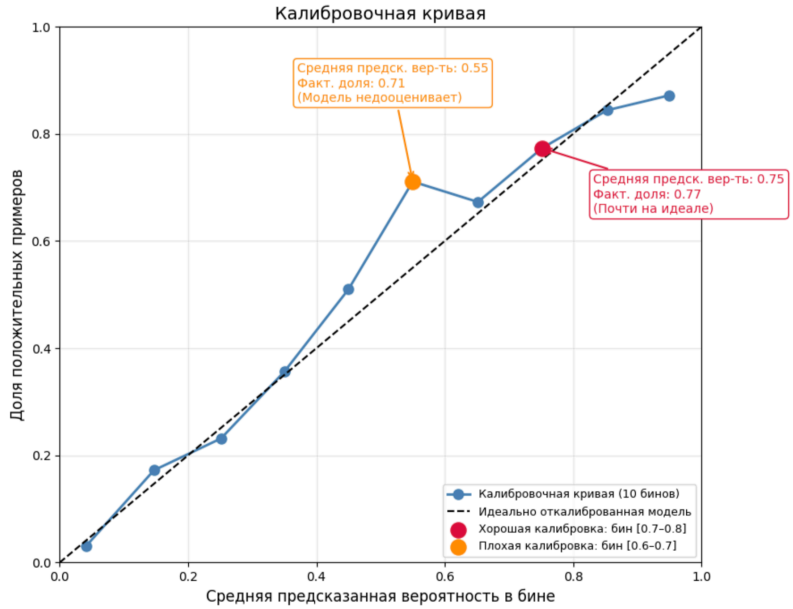


Рис. 21. Калибровочная кривая.

Красная точка показывает зону отличной работы модели. В диапазоне предсказаний от $0{,}7$ до $0{,}8$ модель выдала среднюю вероятность $0{,}75$. Если мы посмотрим на реальность, то увидим, что событие действительно произошло в 77% случаев. Разрыв между ожиданием и реальностью минимален — модель здесь работает почти идеально, и её прогнозам можно доверять.

Совсем другая ситуация складывается в зоне, отмеченной оранжевой точкой. Здесь модель выдала среднюю предсказанную вероятность $0{,}55$, однако на самом деле событие наступило в 71% случаев (фактическая доля $0{,}71$). Именно в таких случаях говорят, что **модель недооценивает** вероятность.

Рассмотрим основные методы калибровки.

6.1. СИГМОИДНАЯ КАЛИБРОВКА ПЛАТТА (PLATT SCALING)

Метод Платта хорошо работает, если модель предсказывает вероятности, которые слишком уверены (например, близки к 0 или 1), и их необходимо «сгладить». Этот метод особенно полезен для моделей, таких как SVM или решающие деревья, у которых нет встроенной калибровки вероятностей. Он предполагает подгонку логистической функции (сигмоиды) к предсказанным значениям модели.

Формула:

$$P(y = 1 \mid \mathbf{x}) = \frac{1}{1 + \exp(Af(\mathbf{x}) + B)},$$

где A и B — параметры, подбираемые на валидационной подвыборке, $f(\mathbf{x})$ — ответ исходной модели.

То есть по сути ответ модели просто подается на вход логистической регрессии, которая его перевзвешивает.

Рассмотрим пример. Мы обучаем SVM и предсказываем вероятности, наивно пропуская ответы модели через сигмоиду, а затем строим калибровочную кривую и распределение предсказанных вероятностей.

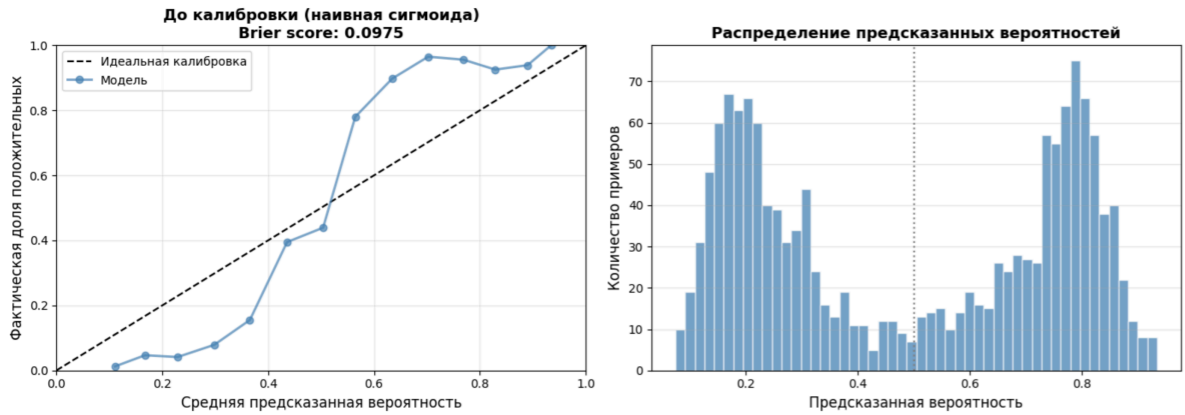


Рис. 22. Калибровочная кривая и ответы SVM до калибровки.

Как можно видеть, предсказания более-менее равномерно распределены по всему диапазону, особенно много точек в области $0{,}1$–$0{,}3$ и $0{,}7$–$0{,}9$, что не соответствует идеальной калибровке, поэтому не позволяет интерпретировать ответы модели как вероятности принадлежности к классу 1.

Если же мы применим калибровку Плата к модели, то получим уже несколько другую картину:

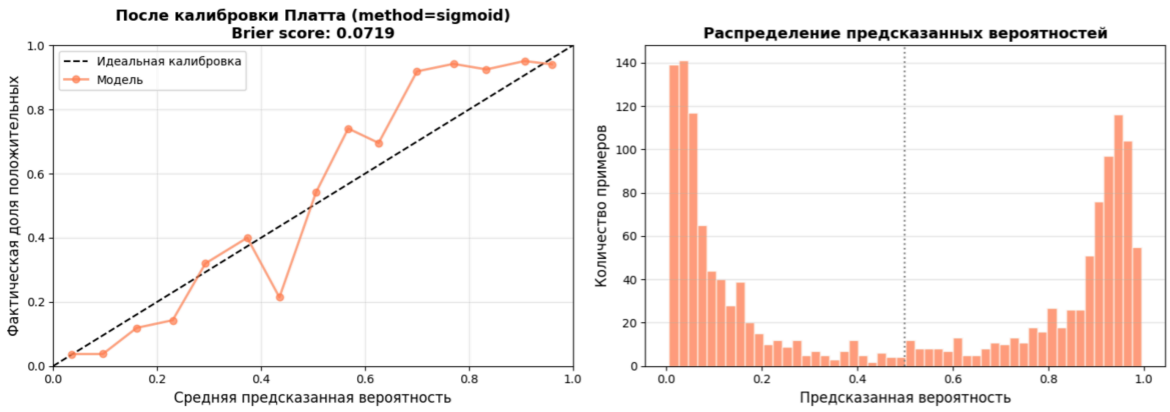


Рис. 23. Калибровочная кривая и ответы SVM после калибровки.

Теперь модель стала более уверенно относить объекты к классу 0 и 1. А также калибровочная кривая стала меньше отдаляться от идеальной калибровки. Тем не менее калибровка все еще не идеальная, и это нормально. Также снизился brier score (чем ниже, тем лучше), он позволяет оценивать качество калибровки.

Реализация в sklearn по умолчанию предполагает наличие кросс-валидации. То есть выборка разделяется на n частей, на каждой $n-1$ части обучается основная модель, а на 1 части обучается калибровка. В итоге получается ансамбль из n моделей, где предсказания каждой модели усредняются для получения итогового предсказания.

```python
from sklearn.svm import SVC
from sklearn.calibration import CalibratedClassifierCV

base_model = SVC()
calibrated_model = CalibratedClassifierCV(base_model,
                                          method='sigmoid', # Для калибровки Платта
                                          cv=5) # Делим выборку на 5 фолдов

calibrated_model.fit(X_train, y_train)
```

6.2. ИЗОТОНИЧЕСКАЯ РЕГРЕССИЯ

Изотоническая регрессия — более гибкий метод, который не предполагает конкретной формы калибровочной функции. Единственное ограничение: функция должна быть монотонно неубывающей.

Математически задача выглядит так: дан вектор предсказаний $f(\mathbf{x}_1), \dots, f(\mathbf{x}_n)$ и соответствующие бинарные метки $y_1, \dots, y_n$. Требуется найти монотонную функцию $g(\cdot)$, минимизирующую среднеквадратичную ошибку:

$$\min_{g \text{ монотонна}} \sum_{i=1}^{n} (g(f(\mathbf{x}_i)) - y_i)^2 .$$

Результат — ступенчатая функция, так как минимизирующая MSE монотонная функция на выборке всегда будет кусочно-постоянной (ступенчатой). Ступеньки появляются там, где данные сгруппированы по одинаковым выходам модели или схожим значениям.

Изотоническая регрессия хорошо работает, когда данных много и вероятность действительно сильно изменяется в зависимости от признаков. Однако при малом числе примеров она может переобучиться, так как не сглаживает шум.

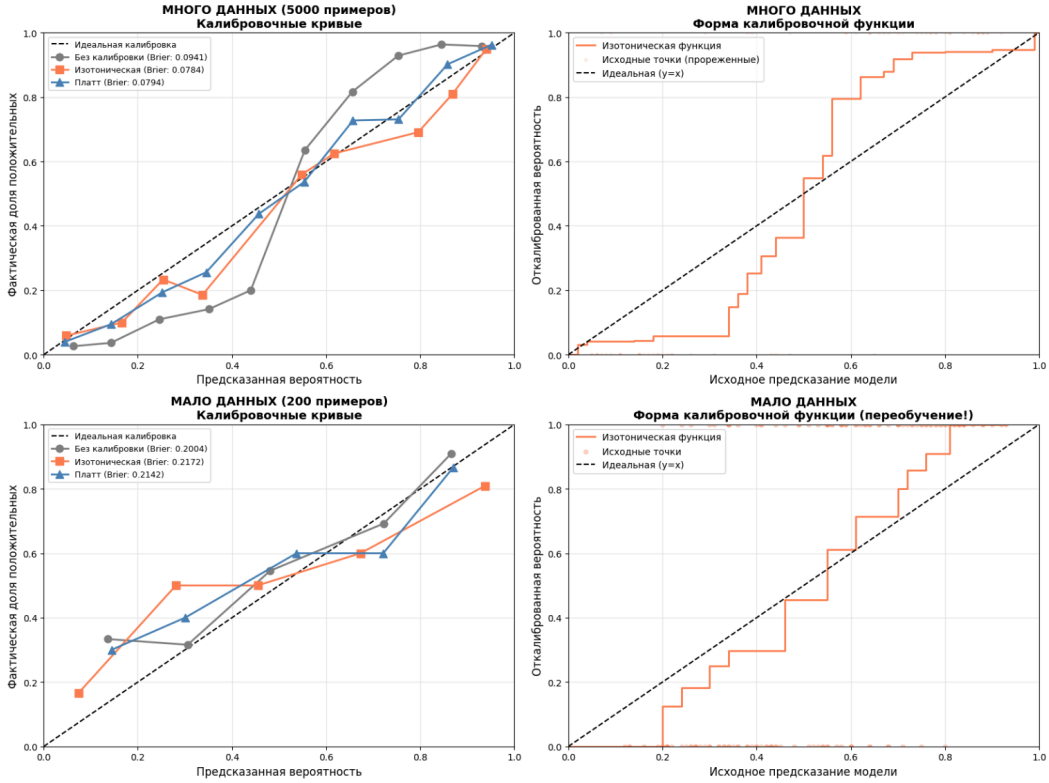


Рис. 24. Сравнение метода Платта и изотонической регрессии для калибровки при разном количестве данных.

На полученных графиках чётко видно, как количество данных влияет на качество калибровки двумя разными методами.

При большом количестве данных (5000 примеров, верхний ряд) оба метода работают хорошо. Изотоническая регрессия создаёт калибровочную функцию с множеством маленьких ступенек, которые плавно аппроксимируют истинную зависимость между предсказаниями модели и реальными вероятностями. Brier score для изотонической регрессии ($0{,}0784$) и метода Платта ($0{,}0794$) практически одинаковы и значительно лучше, чем у неоткалиброванной модели ($0{,}0941$). Здесь изотоническая регрессия **не переобучается** — у неё достаточно данных, чтобы надёжно оценить каждую ступеньку, и функция получается гладкой и устойчивой.

При малом количестве данных (200 примеров, нижний ряд) ситуация меняется. Изотоническая регрессия создаёт калибровочную функцию всего с несколькими большими ступеньками, которые делают резкие скачки — например, с $0{,}1$ сразу до $0{,}5$, затем до $0{,}7$. Это классическое **переобучение**: модель «запоминает» конкретные точки в обучающей выборке вместо того, чтобы выявлять общую закономерность. Brier score изотонической регрессии ($0{,}2172$) становится **хуже**, чем у метода Платта ($0{,}2142$), хотя оба метода всё равно хуже неоткалиброванной модели ($0{,}2004$). Метод Платта, будучи параметрическим (всего два параметра A и B в сигмоиде), действует как регуляризация — его гладкая форма не позволяет подстроиться под шум в данных.

Вывод: изотоническая регрессия — мощный инструмент для калибровки, но только при достаточном количестве данных. При малых выборках её гибкость оборачивается переобучением, и метод Платта показывает более стабильные результаты благодаря своей параметрической форме. Реализация в sklearn (всё то же самое отличие в аргументе `method`)

```python
from sklearn.svm import SVC
from sklearn.calibration import CalibratedClassifierCV

base_model = SVC()
calibrated_model = CalibratedClassifierCV(base_model,
                                          method='isotonic', # Для изотонической
                                           ↪  калибровки
                                          cv=5) # Делим выборку на 5 фолдов

calibrated_model.fit(X_train, y_train)
```

6.3. МЕТРИКИ ОЦЕНКИ ПРИЁМОВ КАЛИБРОВКИ ВЕРОЯТНОСТЕЙ

Чтобы численно оценить, насколько хорошо модель откалибрована, используют метрики, которые сравнивают предсказанные вероятности с истинными метками. Если модель идеально откалибрована и имеет высокую разрешающую способность, то для всех объектов с предсказанной вероятностью p доля положительных среди них равна p, и тогда значения этих метрик достигают своего минимума. Рассмотрим две основные.

1. **Log loss**
   Это основная метрика для вероятностных моделей классификации. Измеряет, насколько «уверенно» и точно модель предсказывает вероятности. Штраф зависит от того, насколько предсказанная вероятность p_i близка к истинной метке y_i:

$$\text{LogLoss} = -\frac{1}{n}\sum_{i=1}^{n}\left[y_i \log(p_i) + (1-y_i)\log(1-p_i)\right],$$

   где n — число объектов; $y_i \in \{0,\ 1\}$ — истинная метка, $p_i \in [0,\ 1]$ — предсказанная моделью вероятность класса 1.
   Связь с калибровкой: если модель систематически завышает или занижает вероятности (плохо откалибрована), log loss резко возрастает, особенно когда модель выдаёт уверенные, но неверные предсказания (например, $p = 0{,}99$ для объекта класса 0). Поэтому минимизация log loss стимулирует модель выдавать вероятности, близкие к истинным частотам, что и означает хорошую калибровку. То есть, дополнительно калибруя модель, мы снижаем уверенность её неверных предсказаний, снижая log loss.

2. **Brier score**
   Измеряет среднеквадратичное отклонение предсказанной вероятности от истинной метки:

$$\text{Brier} = \frac{1}{n}\sum_{i=1}^{n}(p_i - y_i)^2.$$

   Связь с калибровкой: для идеально откалиброванной модели Brier score принимает минимальное возможное значение, равное доле дисперсии, которая не может быть объяснена (неопределённость данных). Чем ближе Brier score к этому минимуму, тем лучше вероятности отражают реальные частоты.

Отличие: log loss сильнее штрафует за «уверенные ошибки», поэтому он более чувствителен к грубым нарушениям калибровки на краях диапазона вероятностей. Brier score даёт более сбалансированную

оценку, но менее резко реагирует на единичные экстремальные промахи.

При этом важно отметить, что **метрики и калибровочная кривая — не одно и то же.** Калибровочная кривая показывает где именно модель ошибается: в каких бинах вероятностей она систематически завышает или занижает предсказания. Log loss и Brier score дают единую числовую меру качества, агрегируя ошибки по всем бинам. Поэтому на практике их используют вместе: кривая — для визуальной диагностики (например, модель стабильно переоценивает малые вероятности), метрики — для формального сравнения разных моделей или настроек калибровки.

6.4. ПРИМЕНЕНИЕ КАЛИБРОВКИ

Для наглядности рассмотрим задачу прогнозирования дефолта по кредитной карте (подробнее о датасете). Была обучена логистическая регрессия, затем её предсказания были откалиброваны с помощью изотонической регрессии. Ниже представлены визуализация калибровочных кривых (см. рисунок 25) и сравнительная таблица метрик.



Рис. 25. Применение калибровки на примере датасета.

Модель	Brier score	Log loss	AUC-ROC
Логистическая регрессия	0,146000	0,469800	0,723900
Изотоническая калибровка	0,140400	0,449400	0,718500
Разница (%)	3,988604	4,539386	−0,745959

В этом случае Brier score и log loss снизились на ~4%, что говорит о более точных вероятностных предсказаниях благодаря калибровке. Но качество прогноза по AUC-ROC итоговой модели стал чуть хуже. Это связано с тем, что калибровщик обучается на отложенной валидационной подвыборке и далеко не факт, что она хорошо совпадает с тестовой подвыборкой, на котором проводили финальное тестирование. Таким образом, качество может слегка ухудшиться.

Если кратко

- → Логистическая регрессия штрафует за уверенные ошибки (логарифмическая функция потерь); даёт честную вероятность: предсказание $0{,}8$ означает, что среди таких объектов примерно 80% действительно относятся к такому классу.
- → SVM (метод опорных векторов) ищет границу с максимальным отступом между классами; использует опорные объекты (ближайшие к границе); не выдаёт честных вероятностей (расстояние до границы $\neq$ вероятность); лучше справляется с выбросами, чем логистическая регрессия.
- → Ядерный трюк помогает решить задачи классификации, в которых данные линейно неразделимы: вместо прямого отображения в пространстве большей размерности, где объекты становятся линейно разделимыми, подставлять функцию $K(\mathbf{x}_i, \mathbf{x}_j)$, которая ведёт себя так, будто мы работаем в пространстве высокой размерности.
- → Калибровка вероятностей проверяет, насколько модель честна в вероятностях; позволяет получить вероятности для моделей, если их нет (SVM, деревья). Методы: сигмоида Платта (логистическая кривая, даёт гладкое приближение), изотоническая регрессия (строит монотонную функцию, даёт ступенчатый результат, риск переобучения).